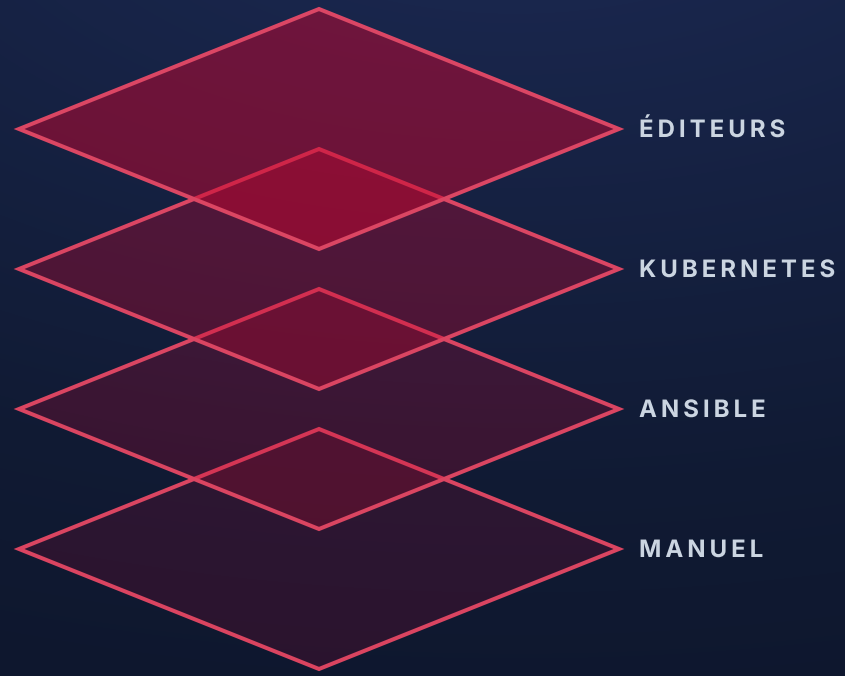
COMPRENDRE OPENSTACK · MODULE 5

Déployer OpenStack : panorama et critères de choix

Outils communautaires, distributions d'éditeurs, offres gérées — comment ils fonctionnent et comment choisir



DevStack · Kolla-Ansible · OpenStack-Ansible · OpenStack-Helm · Sunbeam · RHOSO · MOSK

État des outils vérifié en octobre 2026 · version de référence : OpenStack 2026.2 « Hibiscus » · exemples Kolla-Ansible et OSA en série 2026.1, ces outils publiant leur 2026.2 avec décalage

INTRODUCTION

Objectif et déroulé

Objectif : savoir situer chaque méthode de déploiement d'OpenStack — ce qu'elle automatise, sur quoi elle s'appuie, ce qu'elle exige de l'équipe — et disposer d'une démarche pour choisir.

À l'issue de ce module, vous saurez :

- Décomposer un déploiement en **couches** et dire quel outil couvre laquelle.
- Classer les méthodes selon trois axes : **forme de livraison**, **orchestrateur**, **socle d'exécution**.
- Décrire le fonctionnement de DevStack, Kolla-Ansible, OpenStack-Ansible, OpenStack-Helm, Sunbeam, RHOSO et MOSK.
- Distinguer outil communautaire, distribution supportée et offre gérée.
- **Comparer** les méthodes sur l'exploitation : montée de version, personnalisation, fonctionnement déconnecté.
- Appliquer une **grille de critères** et un arbre de décision à un contexte donné.

1 · Ce que « déployer » veut dire
Couches, cycle de vie, axes de classement, paysage et statut des outils.

2 · Apprendre et développer
DevStack, installations tout-en-un, installation manuelle.

3 · Outils communautaires de production
Kolla-Ansible, OpenStack-Ansible, OpenStack-Helm et dérivés, provisionnement.

4 · Distributions et offres gérées
Canonical, Red Hat, Mirantis et les autres ; modes de consommation.

5 · Comparaison transversale
Technique, exploitation, compétences, déconnecté, échelle.

6 · Critères de choix
Dix critères, arbre de décision, profils types, grille de notation.

01

Ce que « déployer OpenStack » veut dire

Installer les services n'est qu'une étape. Une méthode de déploiement se juge sur tout ce qu'elle prend en charge, de la machine nue jusqu'à la dixième montée de version.

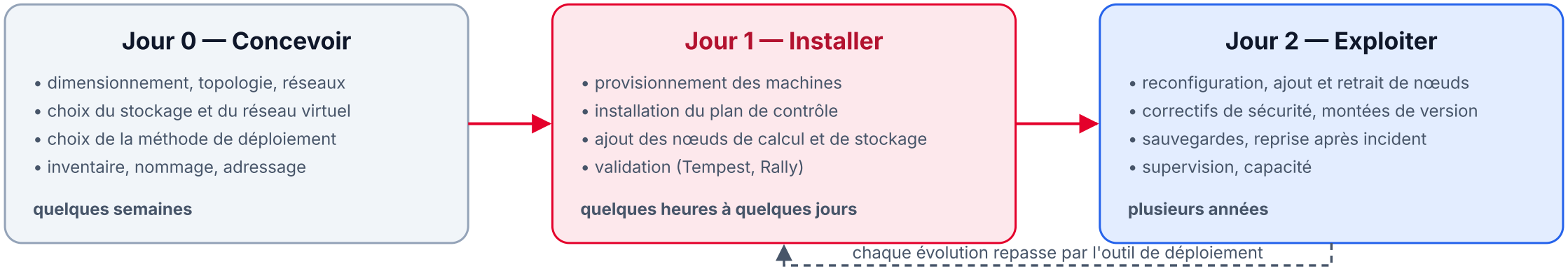
- Les six couches d'un déploiement
- Trois axes pour classer les méthodes
- Trois générations d'outils
- Avant, pendant, après : le cycle de vie
- Formes de livraison et socles d'exécution
- Carte du paysage et statut des projets

Les six couches d'un déploiement

Couche	Ce qu'il faut faire	Qui s'en charge, selon la méthode
6 · Configuration du cloud ce que voient les utilisateurs	réseaux externes, gabarits, images, quotas, projets, types de volumes	vous, avec Terraform ou la CLI ; amorcée par certains outils (sunbeam configure)
5 · Services OpenStack Keystone, Nova, Neutron, Cinder, Glance…	installer, configurer, initialiser les bases, enregistrer les points d'accès, démarrer	**cœur de métier de tous les outils** Kolla-Ansible, OSA, Helm, Sunbeam, RHOSO…
4 · Services d'infrastructure base de données, bus, répartiteur, cache, stockage	MariaDB Galera, RabbitMQ, HAProxy, Memcached, Ceph, OVN — en haute disponibilité	inclus dans la plupart des outils ; Ceph souvent déployé à part
3 · Préparation des hôtes système prêt à recevoir les services	paquets, noyau, horloge, interfaces et ponts, moteur de conteneurs ou Kubernetes	en partie : bootstrap-servers (Kolla), setup-hosts (OSA), prepare-node (Sunbeam)
2 · Provisionnement des machines du serveur nu au système installé	amorçage PXE, installation du système, configuration des disques et du réseau	outil séparé : MAAS, Ironic / Bifrost, Metal3, Kayobe, ou à la main
1 · Matériel et réseau physique baies, câblage, commutateurs, adressage	conception, achat, mise en baie — toujours à votre charge, sauf offre hébergée	aucun outil de déploiement

Première question à poser à une méthode : à quelle couche commence-t-elle, à laquelle s'arrête-t-elle ? Deux outils « de déploiement » peuvent ne couvrir que la couche 5 pour l'un, les couches 2 à 6 pour l'autre.

Avant, pendant, après : le déploiement est un cycle



Ce que cela change dans le choix

- Une démonstration juge le **jour 1** ; l'exploitation se joue au **jour 2**. Une installation en vingt minutes ne dit rien de la montée de version dans deux ans.
- L'outil choisi au départ devient le **seul chemin** pour modifier le cloud : toute intervention manuelle hors outil sera écrasée à la prochaine exécution.
- Changer de méthode en cours de vie revient presque toujours à **reconstruire** un cloud à côté et à migrer les charges.

Opération du jour 2	Question à poser à l'outil
Changer une option	Une commande, ou un redéploiement complet ?
Ajouter un hyperviseur	Sans toucher aux nœuds existants ?
Monter de version	Sans interruption des API ? Saut de version possible ?
Remplacer un contrôleur	Procédure documentée et testée ?
Revenir en arrière	Prévu, ou restauration de sauvegarde ?

1 · CE QUE « DÉPLOYER » VEUT DIRE

Trois axes pour classer les méthodes

Axe 1 · Forme de livraison

Sous quelle forme le code d'OpenStack arrive sur les machines.

Options : paquets de la distribution · sources dans un environnement Python · images de conteneurs · snaps

Axe 2 · Orchestrateur

Quel moteur enchaîne les étapes, applique la configuration et la maintient.

Options : scripts shell · Ansible · Puppet · Juju · opérateurs Kubernetes et Helm

Axe 3 · Socle d'exécution

Sur quoi tourne le plan de contrôle une fois installé.

Options : directement sur l'hôte · conteneurs Docker ou Podman · conteneurs LXC · pods Kubernetes

Méthode	Forme de livraison	Orchestrateur	Socle du plan de contrôle
DevStack	Sources (git)	Scripts shell	Hôte, services systemd
Installation manuelle	Paquets de la distribution	L'administrateur	Hôte
Kolla-Ansible	Images de conteneurs Kolla	Ansible	Conteneurs Docker ou Podman
OpenStack-Ansible	Sources dans des environnements Python	Ansible	Conteneurs LXC, ou hôte
OpenStack-Helm et dérivés	Images de conteneurs (LOCI)	Helm, opérateurs	Pods Kubernetes
Sunbeam (Canonical)	Images OCI (rocks) et snaps	Juju, Terraform	Pods Kubernetes ; snaps pour le calcul
RHOSO (Red Hat)	Images de conteneurs	Opérateurs OpenShift, Ansible	Pods OpenShift ; RHEL pour le calcul
MOSK (Mirantis)	Images de conteneurs	Opérateur, Helm	Pods sur le Kubernetes de Mirantis

Ces trois choix déterminent presque tout le reste : compétences nécessaires, façon de monter de version, facilité à corriger un service, comportement en environnement déconnecté.

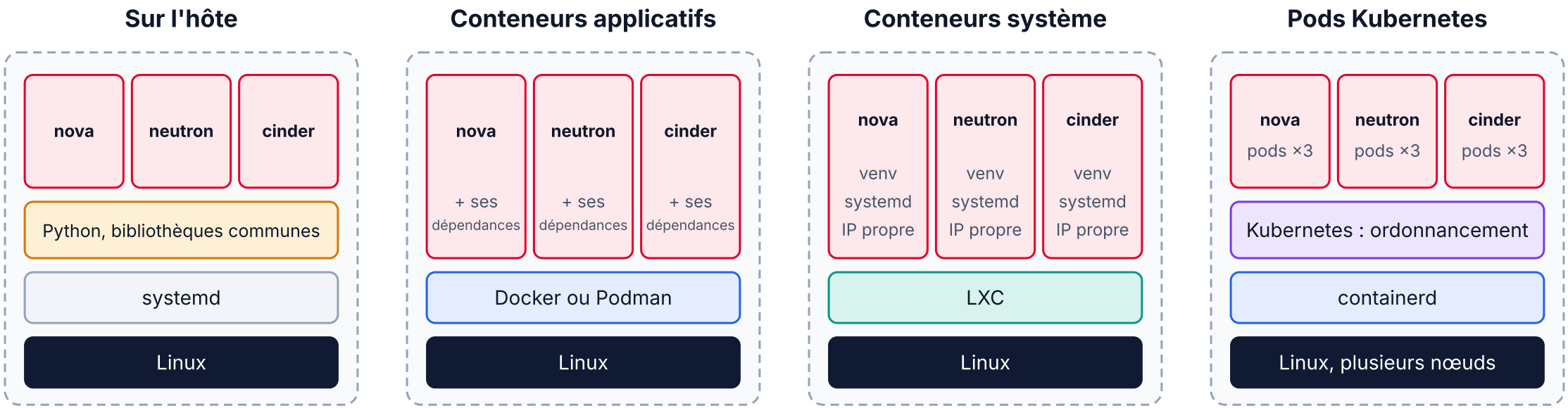
Axe 1 — Les formes de livraison du code

Forme	Principe	Atouts	Limites	Utilisée par
Paquets de la distribution	Paquets `.deb` ou `.rpm` construits par Ubuntu, Debian, RDO	Intégration au système, correctifs par le gestionnaire de paquets	Dépendances Python partagées entre services ; version liée à la distribution	Installation manuelle, Puppet, anciennes distributions
Sources dans un environnement Python	Le code est récupéré par git et installé dans un *virtualenv* par service	Toute version, tout correctif amont ; services isolés les uns des autres	Compilation sur place, plus lente ; dépôt de sources à mettre à disposition	DevStack, OpenStack-Ansible
Images de conteneurs	Une image par service, avec toutes ses dépendances, tirée d'un registre	Identique sur tous les nœuds ; mise à jour = changer d'image ; retour arrière simple	Registre à opérer ; images à reconstruire pour chaque correctif	Kolla-Ansible, OpenStack-Helm, RHOSO, MOSK
Snaps	Paquet autonome et confiné, mis à jour par canaux	Installation en une commande, mises à jour transactionnelles avec retour arrière	Écosystème Ubuntu ; moins de prise sur le contenu	Sunbeam (hyperviseur, MicroCeph, CLI)

Tendance générale : la livraison par images de conteneurs s'est imposée pour le plan de contrôle. Elle supprime le problème historique des dépendances Python incompatibles entre services installés sur une même machine.

Deux familles d'images : les images **Kolla** (un service prêt à l'emploi par image, bases Ubuntu, Debian, Rocky) et les images **LOCI** (plus légères, utilisées par OpenStack-Helm). Les éditeurs publient leurs propres images, signées et supportées.

Axe 3 — Où tourne le plan de contrôle



	Sur l'hôte	Conteneurs applicatifs	Conteneurs système (LXC)	Pods Kubernetes
Isolation des dépendances	Aucune	Par service	Par service	Par service
Redémarrage après panne	systemd, sur le même hôte	Moteur de conteneurs, même hôte	systemd dans le conteneur	Replanifié sur un autre nœud
Diagnostic	Outils Linux habituels	`docker logs`, `docker exec`	Comme une machine virtuelle	`kubectl` ; couche supplémentaire à maîtriser
Exemples	DevStack, guide d'installation	Kolla-Ansible	OpenStack-Ansible	OpenStack-Helm, Sunbeam, RHOSO, MOSK

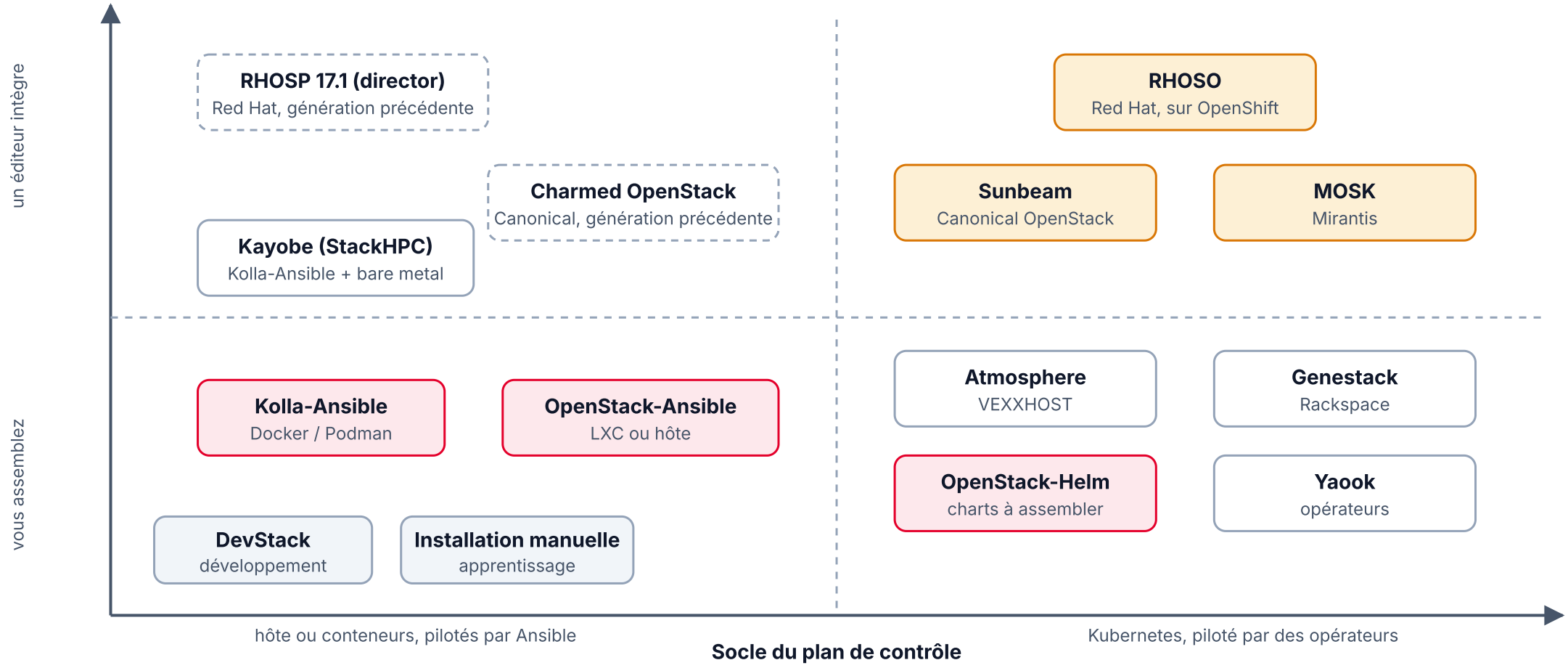
Trois générations d'outils



Cadres en pointillés : projets retirés ou en fin de vie. TripleO et Chef OpenStack ne figurent plus parmi les projets officiels ; Fuel a disparu depuis longtemps.

Les générations coexistent : Kolla-Ansible et OpenStack-Ansible restent très utilisés et activement maintenus. La troisième génération domine chez les éditeurs, pas encore dans toutes les équipes d'exploitation.

Carte du paysage



Lecture : en bas, des outils libres que l'on assemble et exploite soi-même ; en haut, des distributions intégrées et supportées. À gauche, la filière Ansible ; à droite, la filière Kubernetes, vers laquelle les éditeurs ont tous basculé.

Statut des projets en 2026

Outil	Porté par	Statut	À retenir
DevStack	Communauté OpenStack (assurance qualité)	**Actif**	Outil de développement et de test ; sert à l'intégration continue du projet
Kolla, Kolla-Ansible	Projet officiel OpenStack	**Actif**	Suit chaque version ; hôtes Ubuntu 24.04, Debian 13, Rocky Linux 10, CentOS Stream 10 (sans images Kolla)
OpenStack-Ansible	Projet officiel OpenStack	**Actif**	Suit chaque version ; Ubuntu, Debian, Rocky, CentOS Stream
OpenStack-Helm	Projet officiel OpenStack	**Actif**	Collection de charts ; base de plusieurs distributions sur Kubernetes
Sunbeam	Projet officiel OpenStack, mené par Canonical	**Actif**	Base de Canonical OpenStack ; succède à MicroStack
OpenStack Charms	Projet officiel OpenStack, mené par Canonical	Maintenu	Génération précédente de Canonical ; les nouveaux déploiements vont vers Sunbeam
Puppet OpenStack	Projet officiel OpenStack	Maintenu	Modules de configuration, sans outil d'orchestration associé
TripleO	Communauté, Red Hat	**Retiré**	Remplacé chez Red Hat par des opérateurs sur OpenShift (RHOSO)
Chef OpenStack	Communauté	**Retiré**	Dépôts vidés ; plus aucun mainteneur
Packstack	Communauté RDO	Fin de vie	À ne pas retenir pour un nouveau projet ; vérifier son état avant tout usage

Enseignement : un outil de déploiement vit et meurt. La santé du projet — nombre de contributeurs, régularité des versions, diversité des entreprises qui le portent — est un critère de choix à part entière.

Où vérifier : la liste des équipes officielles sur `governance.openstack.org`, l'activité des dépôts sur `opendev.org`, la date de la dernière version sur `releases.openstack.org`.

02

Apprendre et développer

Avant la production, il faut un cloud pour comprendre, tester et former. Ces méthodes privilégient la rapidité et la transparence, pas la robustesse.

- DevStack : le cloud du développeur
- Les installations tout-en-un
- DevStack en pratique
- L'installation manuelle, pas à pas

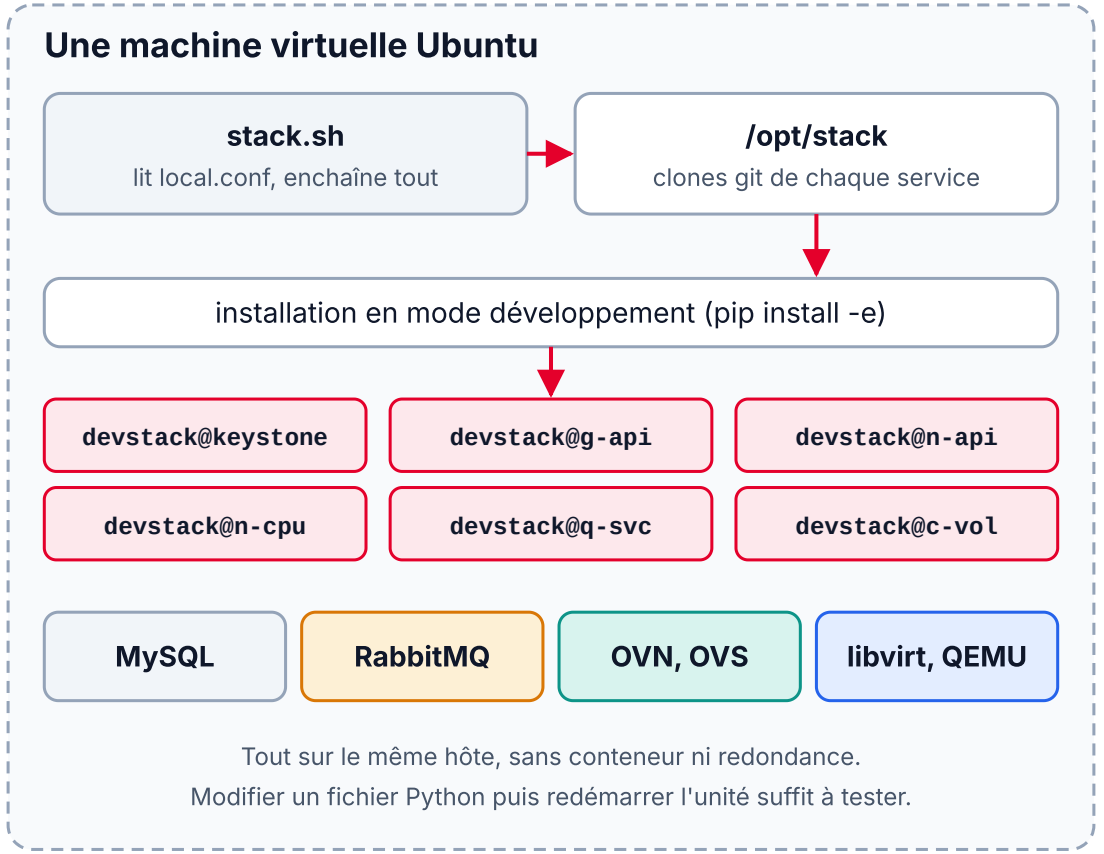
DevStack : le cloud du développeur

Principe : un ensemble de scripts shell qui clone les dépôts de chaque service, installe leurs dépendances et lance tout sur **une seule machine**, à partir des sources.

- **Pour qui :** contributeurs au code d'OpenStack, tests d'intégration, découverte des API.
- **Ce qu'il installe :** Keystone, Glance, Placement, Nova, Neutron (OVN), Cinder (LVM sur fichier), Horizon ; d'autres services par greffons.
- **Comment :** chaque service devient une unité systemd `devstack@<service>` ; les journaux se lisent avec `journalctl`.
- **Atout :** n'importe quelle branche, n'importe quel correctif en cours de revue, en une variable.

Jamais en production : pas de haute disponibilité, mots de passe simples, système hôte profondément modifié, aucune procédure de montée de version. Un redémarrage de la machine peut laisser le cloud inutilisable.

À lancer sur une machine **jetable** : une machine virtuelle dédiée, jamais son poste de travail.



DevStack en pratique

```
# 0. Prérequis : une VM Ubuntu 24.04 neuve, 8 Go de RAM,
#    4 vCPU, 60 Go de disque, accès à Internet

# 1. Créer l'utilisateur dédié
sudo useradd -s /bin/bash -d /opt/stack -m stack
sudo chmod +x /opt/stack
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
sudo -u stack -i

# 2. Récupérer DevStack sur la branche voulue
git clone https://opendev.org/openstack/devstack -b stable/2026.2
cd devstack

# 3. Écrire local.conf (ci-contre), puis lancer
./stack.sh

# 4. Utiliser
source openrc admin admin
openstack service list
```

```
# local.conf
[[local|localrc]]
ADMIN_PASSWORD=secret
DATABASE_PASSWORD=$ADMIN_PASSWORD
RABBIT_PASSWORD=$ADMIN_PASSWORD
SERVICE_PASSWORD=$ADMIN_PASSWORD
HOST_IP=192.168.56.10

# Ajouter un service par greffon
enable_plugin heat https://opendev.org/openstack/heat stable/2026.2
```

Commande	Effet
`./stack.sh`	Installe et démarre tout (20 à 40 minutes)
`./unstack.sh`	Arrête les services
`./clean.sh`	Supprime ce que DevStack a installé
`systemctl restart devstack@n-cpu`	Redémarre un service après modification du code
`journalctl -f -u devstack@n-api`	Suit le journal d'un service

> Plusieurs machines : un nœud « contrôleur » et des nœuds « calcul » qui déclarent `SERVICE_HOST` et la liste réduite `ENABLED_SERVICES`.

Les installations tout-en-un

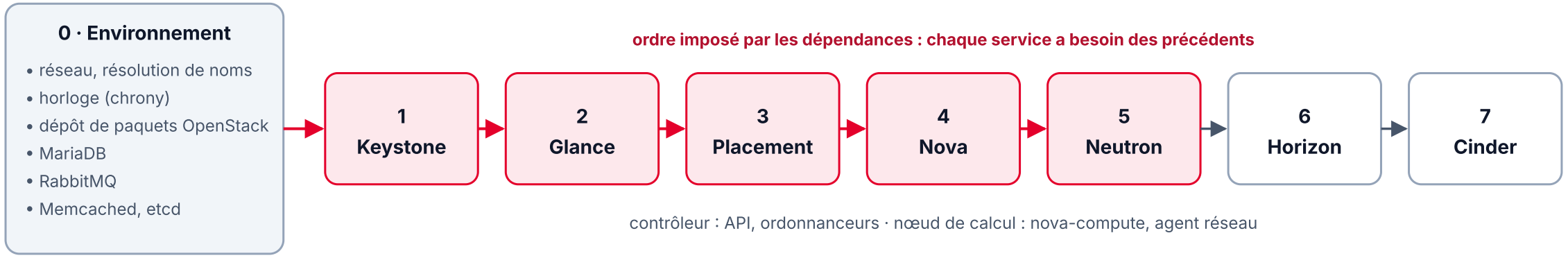
La plupart des outils de production proposent un mode « une seule machine ». On y retrouve les mêmes conteneurs, les mêmes fichiers et les mêmes commandes qu'en production : c'est le meilleur terrain d'apprentissage d'un outil donné.

Méthode	Ressources minimales	Mise en route	Fidélité à la production	Survit à un redémarrage	Usage conseillé
DevStack	8 Go de RAM, 60 Go de disque	`./stack.sh`	Faible : services sur l'hôte, sources	Non garanti	Développement, lecture du code
Kolla-Ansible « all-in-one »	8 Go de RAM, 40 Go, 2 interfaces	Inventaire `all-in-one`, trois commandes	Élevée : mêmes images, mêmes playbooks	Oui	Découvrir Kolla, lab de formation
OpenStack-Ansible « AIO »	16 Go de RAM, 80 Go ou plus	Scripts d'amorçage AIO, puis playbooks	Élevée : mêmes rôles, conteneurs LXC	Oui	Découvrir OSA, tester une configuration
Sunbeam, nœud unique	16 Go de RAM, 4 cœurs, 100 Go ou plus	`sunbeam cluster bootstrap`	Élevée : Kubernetes, Juju, snaps	Oui	Découvrir Canonical OpenStack
Installation manuelle	2 machines virtuelles, 4 à 8 Go chacune	Guide d'installation, service par service	Moyenne : paquets, sans haute disponibilité	Oui	Comprendre chaque composant

Limite commune : aucune haute disponibilité, un seul hyperviseur, stockage sur fichier ou sur disque local. On y apprend l'outil et les API, pas le comportement en panne.

Pour aller plus loin sans matériel : trois à cinq machines virtuelles avec virtualisation imbriquée permettent de monter un vrai plan de contrôle redondant et de simuler la perte d'un nœud.

L'installation manuelle : comprendre avant d'automatiser



Le même enchaînement pour chaque service

1	Créer la base de données et son compte
2	Créer l'utilisateur de service dans Keystone, lui donner le rôle `admin` sur le projet `service`
3	Enregistrer le service et ses points d'accès dans le catalogue
4	Installer les paquets
5	Renseigner le fichier de configuration : base, bus, Keystone
6	Initialiser le schéma (`<service>-manage db sync`)
7	Démarrer et activer les services, puis vérifier

- **Valeur pédagogique irremplaçable :** on voit de ses yeux ce que tout outil automatise — et où chercher quand il échoue.
- **Source :** le guide d'installation officiel, décliné pour Ubuntu, Debian et les distributions de la famille Red Hat.
- **Résultat :** un contrôleur et un nœud de calcul, sans redondance ni chiffrement.

> **Pas une méthode de production :** non reproductible, sans haute disponibilité, et chaque montée de version serait à refaire à la main, service par service, sur chaque machine.

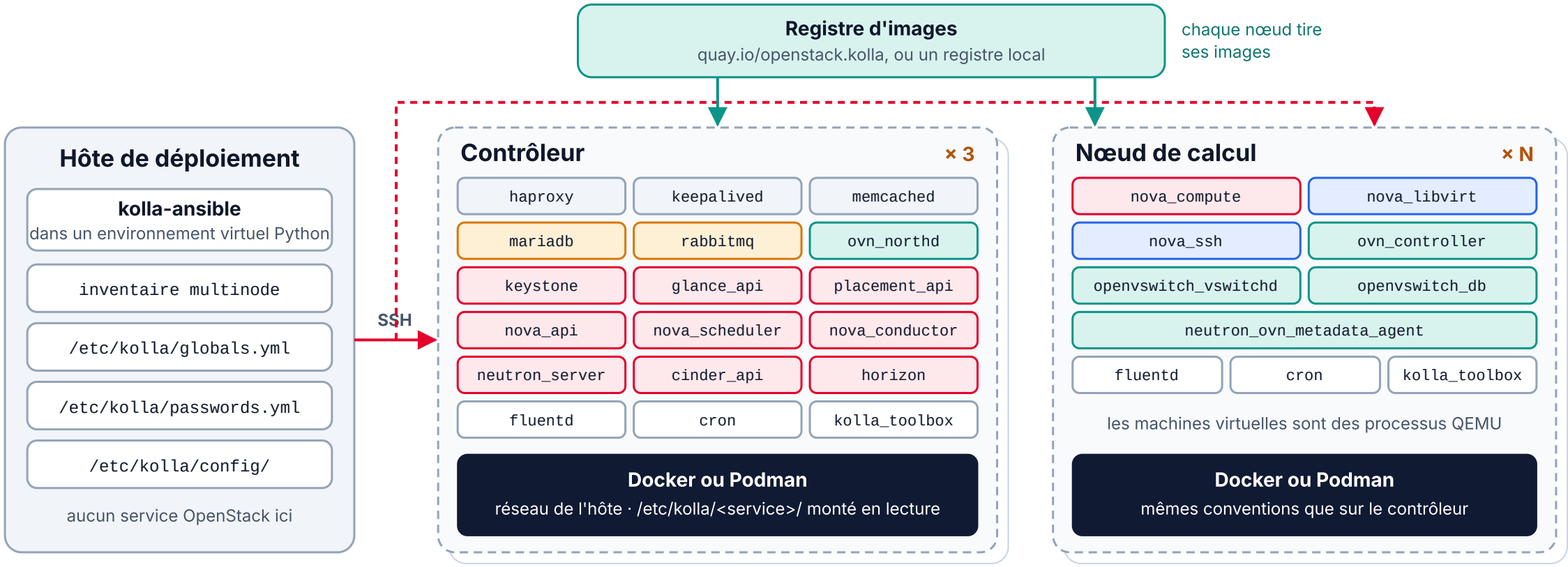
03

Outils communautaires de production

Trois familles officielles, développées sous la gouvernance d'OpenStack, gratuites et sans support contractuel : des conteneurs pilotés par Ansible, des conteneurs système LXC, des charts Helm sur Kubernetes.

- Kolla-Ansible : architecture, commandes, configuration
- OpenStack-Ansible : architecture et déroulé
- OpenStack-Helm et ses dérivés
- Kayobe : Kolla-Ansible depuis la machine nue
- Kolla-Ansible et OpenStack-Ansible face à face
- Briques réutilisables et provisionnement des machines

Kolla-Ansible : un conteneur par processus



À retenir : les images sont construites à l'avance par le projet **Kolla** ; **Kolla-Ansible** se contente de générer la configuration, de tirer les images et de démarrer les conteneurs. Mettre à jour un service, c'est remplacer son image.

Kolla-Ansible : déroulé d'un déploiement

Prérequis, avant toute commande

- Hôtes en **CentOS Stream 10, Debian 13, Rocky Linux 10 ou Ubuntu 24.04** (série 2026.1 ; pas d'images Kolla pour CentOS Stream 10), accessibles en SSH par clé avec élévation de privilèges.
- Par nœud de test : **8 Go** de mémoire, **40 Go** de disque, **deux interfaces réseau**.
- Une **adresse virtuelle** libre sur le réseau interne, pour HAProxy.
- Une interface **sans adresse IP** pour le réseau externe de Neutron.

Après le déploiement

Besoin	Commande
Appliquer une option modifiée	`reconfigure`
Ne traiter qu'un service	`--tags nova`
Ne traiter qu'un hôte	`--limit calcul05`
Télécharger les images à l'avance	`pull`
Monter de version	`upgrade`
Sauvegarder MariaDB	`mariadb-backup`

```
# 1 · Installer l'outil sur l'hôte de déploiement
python3 -m venv ~/kolla-venv
source ~/kolla-venv/bin/activate
pip install -U pip
pip install git+https://opendev.org/openstack/kolla-ansible@stable/2026.1
kolla-ansible install-deps

# 2 · Préparer la configuration
sudo mkdir -p /etc/kolla && sudo chown $USER:$USER /etc/kolla
cp -r ~/kolla-venv/share/kolla-ansible/etc_examples/kolla/* /etc/kolla
cp ~/kolla-venv/share/kolla-ansible/ansible/inventory/multinode .
kolla-genpwd                    # remplit /etc/kolla/passwords.yml
# éditer ./multinode et /etc/kolla/globals.yml

# 3 · Déployer
kolla-ansible bootstrap-servers -i ./multinode  # moteur de conteneurs
kolla-ansible prechecks         -i ./multinode  # ports, VIP, interfaces
kolla-ansible deploy            -i ./multinode
kolla-ansible post-deploy       -i ./multinode  # écrit clouds.yaml
```

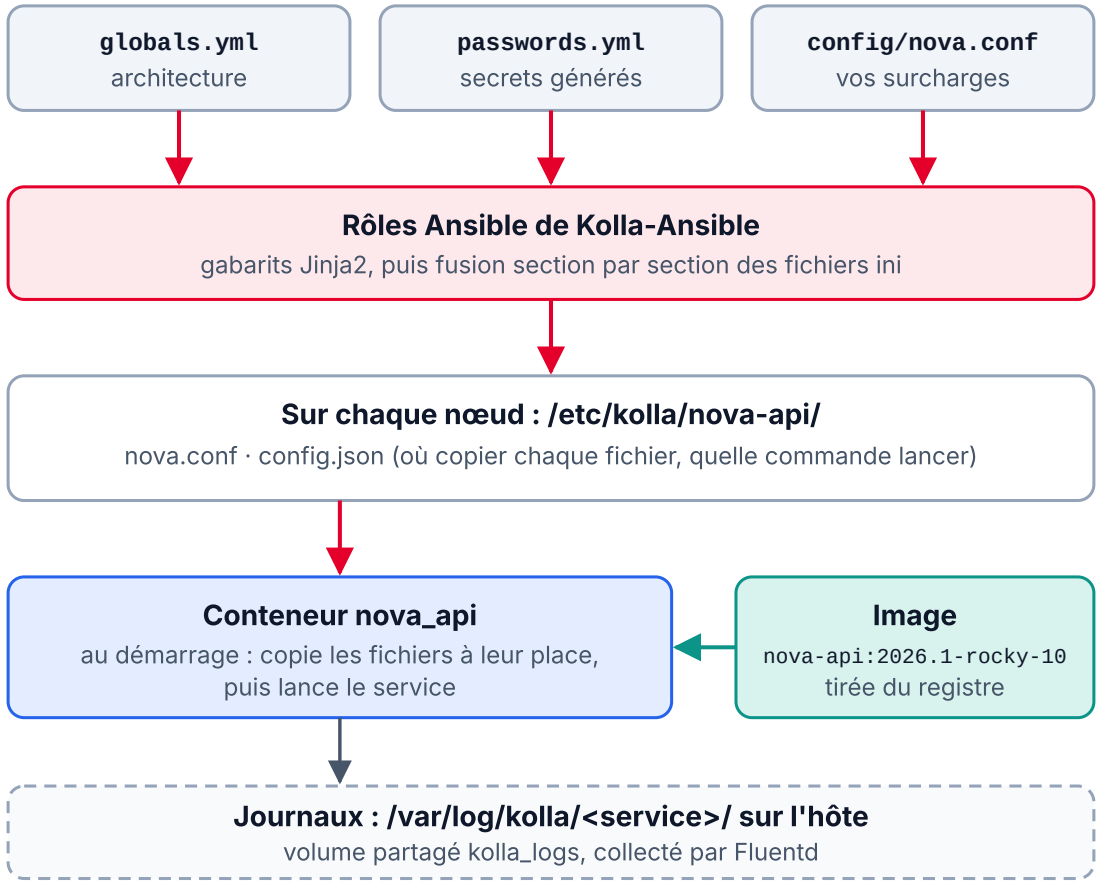
`prechecks` avant chaque `deploy` : il détecte un port déjà occupé, une adresse virtuelle déjà utilisée ou une interface absente avant qu'un conteneur ne soit lancé.

Kolla-Ansible : le modèle de configuration

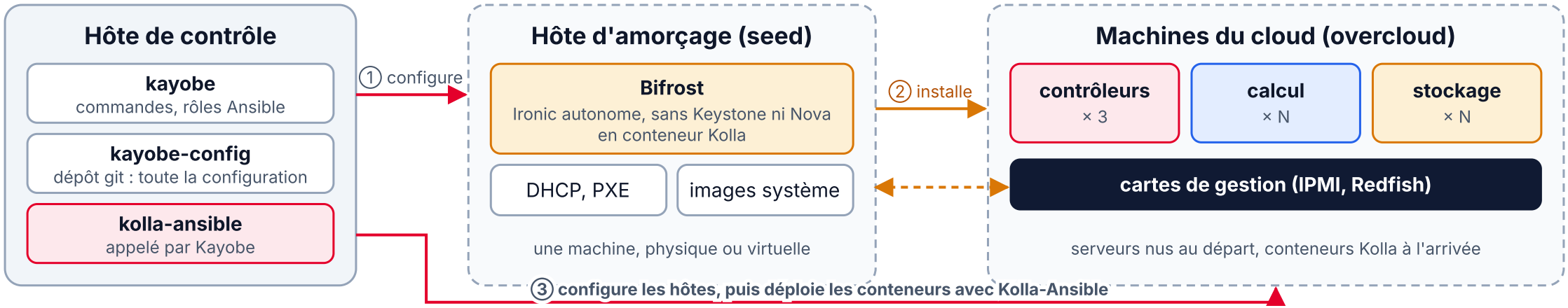
```
# /etc/kolla/globals.yml — extrait
kolla_base_distro: "rocky"
kolla_internal_vip_address: "10.20.0.10"
network_interface: "bond0.20"
neutron_external_interface: "bond1"
neutron_plugin_agent: "ovn"
kolla_enable_tls_external: "yes"
enable_cinder: "yes"
cinder_backend_ceph: "yes"
enable_central_logging: "yes"
```

Trois niveaux de réglage

- `globals.yml` — les choix d'architecture : interfaces, adresse virtuelle, services activés. Quelques dizaines de lignes suffisent.
- `/etc/kolla/config/` — n'importe quelle option d'un service. Un fichier `nova.conf` déposé ici est **fusionné** avec celui que génère l'outil ; un sous-dossier au nom d'un hôte limite la surcharge à cet hôte.
- **Images** — pour un correctif de code ou un pilote tiers, on reconstruit l'image avec `kolla-build` et on la pousse dans son registre.



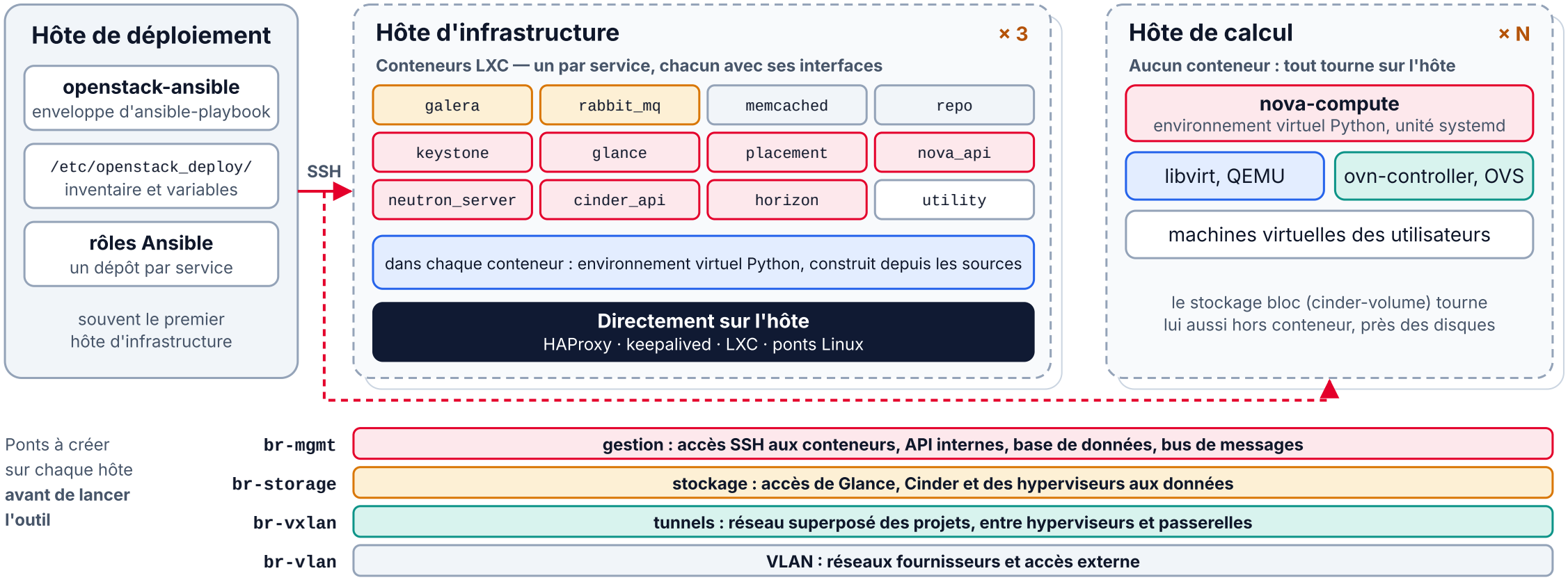
Kayobe : Kolla-Ansible depuis la machine nue




```
kayobe control host bootstrap
kayobe seed host configure
kayobe seed service deploy          # lance Bifrost
kayobe overcloud inventory discover
kayobe overcloud provision          # installe le système
kayobe overcloud host configure     # réseau, disques, noyau
kayobe overcloud service deploy     # Kolla-Ansible
```

- **Ce que Kayobe ajoute :** les couches 2 et 3 — installation du système par PXE, liaisons réseau agrégées, VLAN, partitionnement, pare-feu, utilisateurs.
- **Ce qu'il ne change pas :** les services restent ceux de Kolla-Ansible, avec les mêmes images et les mêmes fichiers de surcharge.
- **Intérêt :** toute l'infrastructure décrite dans **un seul dépôt git**, de la configuration du commutateur d'un port jusqu'à une option de Nova.
- Livré par le projet Kolla, donc versionné avec lui.

OpenStack-Ansible : des services isolés dans des conteneurs LXC



À retenir : OpenStack-Ansible ne télécharge pas d'images. Il **construit** les services depuis les sources au moment du déploiement, dans des conteneurs système qui se comportent comme de petites machines ; un mode sans conteneur (« metal ») existe aussi.

OpenStack-Ansible : déroulé d'un déploiement

Prérequis, avant toute commande

- Hôtes en **Ubuntu 24.04 ou 26.04, Debian 13, CentOS Stream 10 ou Rocky Linux 10** (série 2026.1).
- Hôtes d'infrastructure : au moins **100 Go** de disque, pour les conteneurs et leurs journaux.
- Les **ponts réseau** créés et adressés sur chaque hôte.
- Pour le stockage LVM : un groupe de volumes nommé `cinder-volume`.

```
git clone -b stable/2026.1 \
  https://opendev.org/openstack/openstack-ansible \
  /opt/openstack-ansible
cd /opt/openstack-ansible
scripts/bootstrap-ansible.sh
cp -r etc/openstack_deploy /etc/openstack_deploy
# éditer openstack_user_config.yml, user_variables.yml
scripts/pw-token-gen.py \
  --file /etc/openstack_deploy/user_secrets.yml
openstack-ansible openstack.osa.setup_hosts
openstack-ansible openstack.osa.setup_infrastructure
openstack-ansible openstack.osa.setup_openstack
```

```
# /etc/openstack_deploy/openstack_user_config.yml
cidr_networks:
  management: 172.29.236.0/22
  tunnel:     172.29.240.0/22
  storage:    172.29.244.0/22
shared-infra_hosts:        # Galera, RabbitMQ, Memcached
  infra1: { ip: 172.29.236.11 }
  infra2: { ip: 172.29.236.12 }
  infra3: { ip: 172.29.236.13 }
compute_hosts:
  calcul1: { ip: 172.29.236.21 }
```

Étape	Ce qu'elle fait
`setup_hosts`	prépare les hôtes, crée les conteneurs LXC
`setup_infrastructure`	Galera, RabbitMQ, Memcached, HAProxy, dépôt
`setup_openstack`	construit puis installe chaque service

Le fichier décrit **quels hôtes portent quels groupes** ; l'inventaire détaillé, conteneurs compris, est calculé par l'outil. Montée de version : `scripts/run-upgrade.sh` .

Kolla-Ansible et OpenStack-Ansible face à face

Critère	Kolla-Ansible	OpenStack-Ansible
Forme de livraison	**images de conteneurs construites à l'avance**	code source installé dans des environnements virtuels Python
Isolation	**conteneurs applicatifs Docker ou Podman, réseau de l'hôte**	conteneurs système LXC reliés à des ponts, ou directement sur l'hôte
Un service =	**un conteneur par processus (`nova_api`, `nova_scheduler` …)**	un conteneur par service, plusieurs unités systemd à l'intérieur
Fichiers à écrire	`globals.yml`, `passwords.yml`, **inventaire** `multinode`	`openstack_user_config.yml`, `user_variables.yml`, `user_secrets.yml`
Haute disponibilité	**HAProxy, keepalived, Galera, RabbitMQ en cluster**	mêmes briques ; Galera et RabbitMQ en conteneurs LXC, HAProxy sur l'hôte
Réseau à préparer	**deux interfaces et une adresse virtuelle**	quatre ponts créés et adressés sur chaque hôte
Durée d'un déploiement	**courte : les images sont déjà prêtes**	plus longue : chaque service est construit sur place
Machine nue	**par Kayobe, projet frère**	hors périmètre

Kolla-Ansible convient à qui veut un résultat reproductible et rapide, accepte les choix d'architecture de l'outil et maîtrise Docker ou Podman.

OpenStack-Ansible convient à qui veut garder la main sur chaque service, suivre des branches du code de près et raisonner en « serveurs » plutôt qu'en images.

OpenStack-Helm : OpenStack comme application Kubernetes



Ce que les charts supposent déjà en place — et que vous devez fournir

Kubernetes · réseau de pods (Calico, Cilium…) · contrôleur d'entrée · adresses de service (MetalLB) · stockage persistant (Rook-Ceph ou autre)

OpenStack-Helm : déroulé et points d'attention

Prérequis, avant toute commande

- Un cluster **Kubernetes 1.33 à 1.35** sur **Ubuntu 24.04** : la combinaison testée par le projet pour OpenStack 2025.1 à 2026.1.
- Réseau de pods, contrôleur d'entrée, classe de stockage par défaut, adresses de service.
- `helm` et `kubectl` sur le poste d'administration.

Ordre d'installation des charts

- **Socle :** RabbitMQ, MariaDB, Memcached.
- **Cœur :** Keystone, Heat, Glance, Cinder.
- **Calcul et réseau :** Open vSwitch, libvirt, Placement, Nova, Neutron.
- **Interface :** Horizon.

OpenStack-Helm est une boîte de charts, pas un produit : le cluster, son stockage, ses sauvegardes et l'enchaînement des charts restent à assembler et à maintenir.

```
# Dépôt de charts et greffon d'aide
helm repo add openstack-helm \
  https://tarballs.opendev.org/openstack/openstack-helm
helm plugin install \
  https://opendev.org/openstack/openstack-helm-plugin

# Répartition des rôles par étiquettes
kubectl label --overwrite nodes --all \
  openstack-control-plane=enabled
kubectl label --overwrite nodes --all \
  openstack-compute-node=enabled

# Un service = une installation de chart
helm upgrade --install keystone openstack-helm/keystone \
  --namespace=openstack \
  $(helm osh get-values-overrides -c keystone ${FEATURES})
helm osh wait-for-pods openstack
```

Opération	Avec Helm
Changer une option	modifier les valeurs, `helm upgrade`
Monter de version	nouvelles images et nouveau chart, `helm upgrade`
Revenir en arrière	`helm rollback` — hors schéma de base de données

Sur Kubernetes : les assemblages construits au-dessus des charts

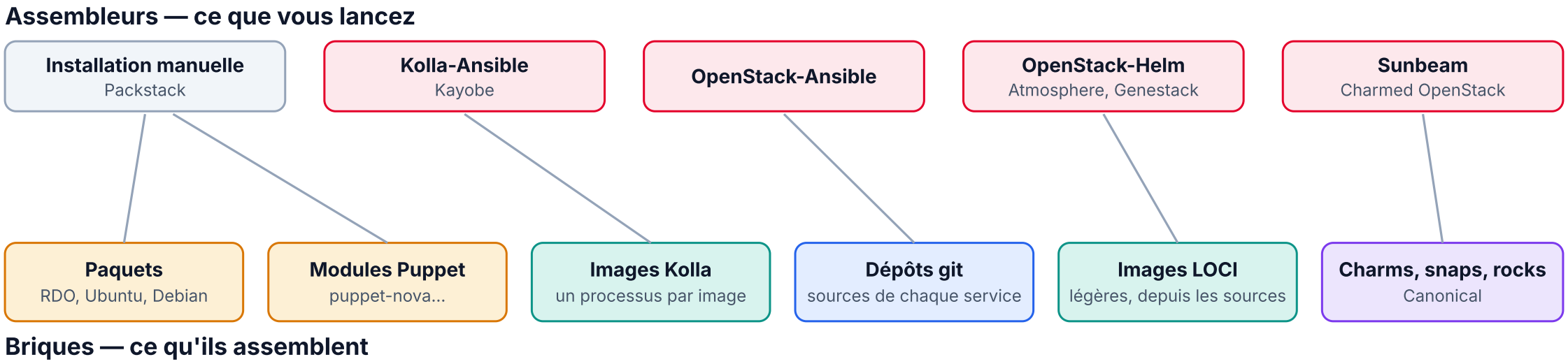
Les charts seuls ne suffisent pas à exploiter un cloud. Plusieurs projets ouverts les complètent par un cluster, un stockage, une supervision et un enchaînement automatisé.

Projet	Porté par	Principe	À regarder si…
Atmosphere	**VEXXHOST**	Ansible installe Kubernetes puis les charts ; Ceph, Keycloak pour l'identité, Prometheus et Loki pour la supervision, Cluster API pour Magnum	vous voulez un ensemble complet et ouvert, avec un support commercial possible
Genestack	**Rackspace**	Kubernetes, charts Helm et surcouches Kustomize ; socle de l'offre OpenStack de Rackspace	vous pratiquez déjà Kustomize et le GitOps
Yaook	**communauté européenne**	des opérateurs Kubernetes dédiés pilotent chaque service, au lieu de charts Helm	vous préférez des opérateurs qui réconcilient en continu
StarlingX	**Open Infrastructure Foundation**	plateforme complète pour sites distants : Kubernetes en socle, OpenStack en application optionnelle	vous déployez de nombreux petits sites en périphérie
MOSK, RHOSO	**Mirantis, Red Hat**	même idée, sous forme de produits supportés — voir la partie 4	vous voulez un contrat de support

Point commun : le plan de contrôle est un ensemble de pods ; Kubernetes se charge du redémarrage, de la répartition et des mises à jour progressives.

Contrepartie : il faut exploiter **deux plateformes**. Une panne de Kubernetes devient une panne d'OpenStack. Genestack et Yaook sont décrits ici d'après leur documentation publique : vérifiez leur état avant de vous engager.

Briques et assembleurs : qui réutilise quoi



Puppet OpenStack

Un module par service, maintenu par une équipe officielle. Sert de fondation à Packstack et a longtemps servi de fondation à TripleO. Pertinent si Puppet gère déjà tout votre parc ; peu de nouveaux déploiements partent de là.

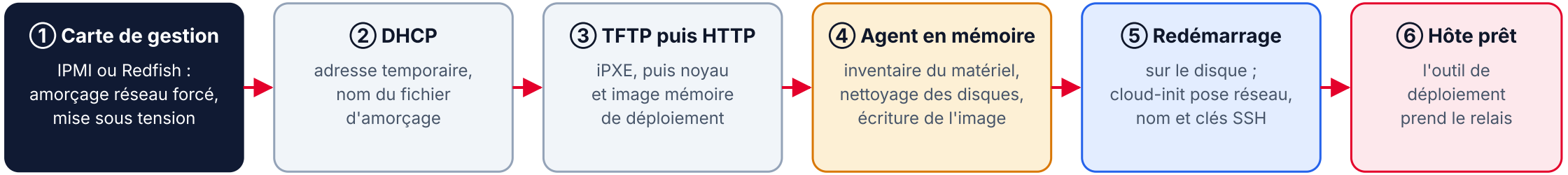
Images Kolla et LOCI

Deux projets officiels qui ne font **que** des images. Kolla : images complètes, par distribution. LOCI : images minimales construites depuis les sources. Réutilisables par votre propre orchestrateur.

Construire son propre outil ?

Possible à partir de ces briques, et pratiqué par de très grands opérateurs. À réserver aux équipes capables de maintenir l'outil pendant dix ans : c'est un produit interne à part entière.

Avant les services : provisionner les machines



Réseau dédié au provisionnement, non routé, sur lequel le serveur d'amorçage est le seul serveur DHCP.

Outil	Principe	Utilisé par	Remarque
MAAS	**service autonome de Canonical : découverte, inventaire, installation, réseau**	Sunbeam, Charmed OpenStack	interface web, API ; Ubuntu d'abord
Ironic autonome, Bifrost	**le service bare metal d'OpenStack, utilisé seul, sans Keystone ni Nova**	Kayobe	même code que celui qu'offrira ensuite le cloud
Metal3	**Ironic piloté par des ressources Kubernetes (`BareMetalHost`)**	RHOSO, plateformes Kubernetes	cohérent si tout est déjà déclaré dans Kubernetes
Installateur existant	**Kickstart, autoinstall, Foreman, images maison**	Kolla-Ansible, OSA, OpenStack-Helm	souvent déjà en place dans l'entreprise : à réutiliser

À trancher tôt : un outil qui « commence au système installé » suppose que vous savez réinstaller quarante serveurs à l'identique. Sans provisionnement automatisé, le remplacement d'un nœud devient une opération manuelle.

Outils communautaires : ce qu'il faut retenir

Kolla-Ansible

- Images prêtes, un conteneur par processus.
- Déploiement et montée de version rapides et reproductibles.
- Configuration courte, surcharges par fichiers.
- Kayobe pour partir de la machine nue.

Limite : sortir des choix d'architecture de l'outil demande de reconstruire des images ou d'écrire ses propres rôles.

OpenStack-Ansible

- Services construits depuis les sources.
- Conteneurs LXC proches d'un serveur classique.
- Très grande liberté de réglage, service par service.
- Suivre une branche ou un correctif est simple.

Limite : préparation réseau exigeante, déploiements plus longs, davantage de variables à comprendre.

OpenStack-Helm

- Plan de contrôle en pods Kubernetes.
- Redémarrage, répartition et mises à jour progressives par la plateforme.
- S'intègre à une chaîne GitOps.
- Base de plusieurs assemblages et produits.

Limite : des charts, pas un produit ; Kubernetes, son réseau et son stockage sont à exploiter en plus.

Dans les trois cas : code gratuit, support communautaire, montées de version à la charge de l'équipe. Les distributions d'éditeurs reprennent souvent ces briques en y ajoutant un contrat.

04

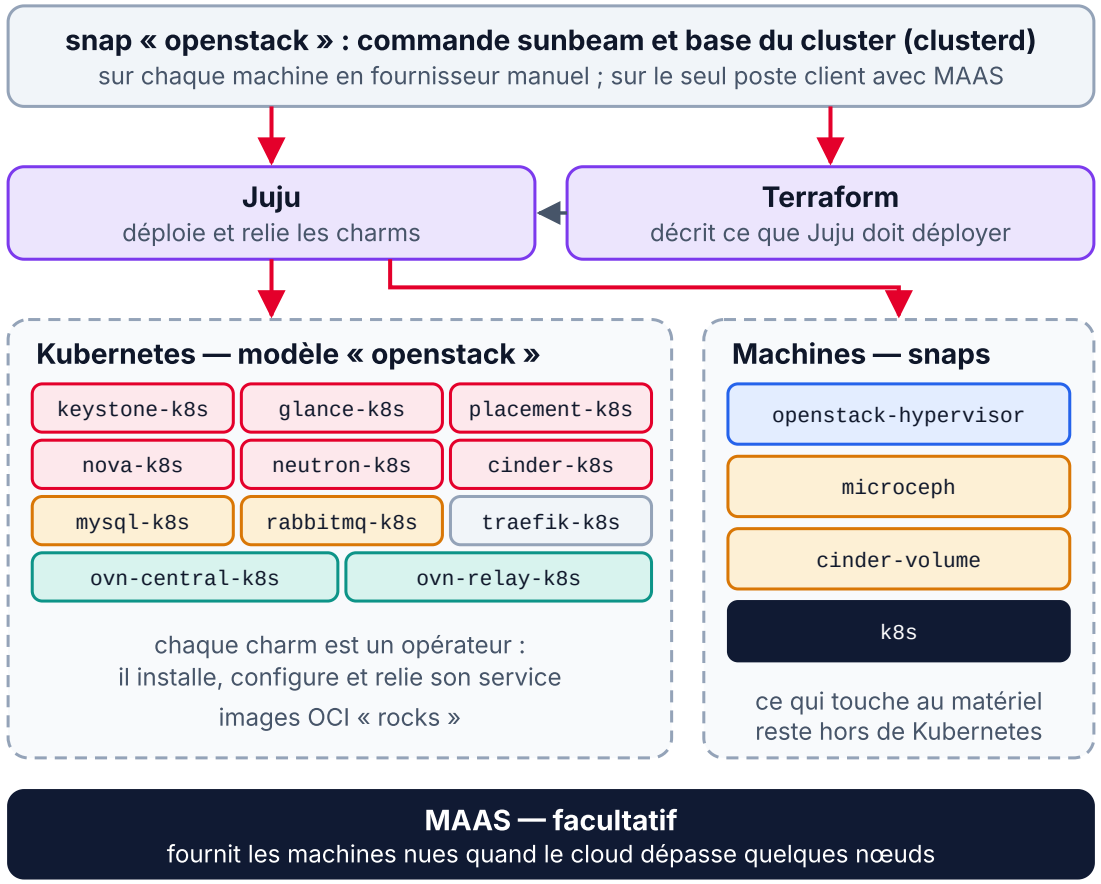
Distributions d'éditeurs et offres gérées

Le même code, mais emballé, testé et maintenu par une entreprise qui s'engage par contrat — ou qui exploite le cloud à votre place.

- Canonical : Sunbeam et Charmed OpenStack
- Mirantis et les autres éditeurs
- Communautaire ou éditeur : ce que l'on achète
- Red Hat : OpenStack Services on OpenShift
- Qui fait quoi : les modes de consommation

 Approfondi au module 6

Canonical : Sunbeam et Charmed OpenStack



	Sunbeam	Charmed OpenStack
Plan de contrôle	**pods Kubernetes**	conteneurs LXD sur des machines
Pilotage	**commande `sunbeam`, qui orchestre Juju**	Juju directement, par lots de charms
Machines nues	**MAAS facultatif**	MAAS dans la pratique
Taille minimale	**une seule machine**	plusieurs machines
Stockage	**MicroCeph**	Ceph, par charms
Réseau	**OVN**	OVN
Position	**voie recommandée pour un nouveau cloud**	maintenu pour les clouds existants

- **« Canonical OpenStack »** est le nom commercial de la distribution fondée sur Sunbeam ; le code de Sunbeam est un projet officiel d'OpenStack.
- **Gratuit sans contrat** : le support s'ajoute par abonnement Ubuntu Pro, sans changer de logiciel.
- **Peu de réglages exposés** : l'outil fait les choix d'architecture à votre place. C'est sa force pour démarrer et sa limite pour les cas atypiques.

4 · DISTRIBUTIONS ET OFFRES GÉRÉES Approfondi au module 6

Sunbeam : déroulé et cycle de vie

Prérequis : Ubuntu Server LTS, un compte avec `sudo`, un nom de machine pleinement qualifié, une plage d'adresses libre pour les points d'accès.

```
# Première machine — le canal fixe la version d'OpenStack
sudo snap install openstack --channel 2026.1/stable
sunbeam prepare-node-script --bootstrap | bash -x
newgrp snap_daemon
sunbeam cluster bootstrap \
  --role control,compute,storage
sunbeam configure --openrc demo-openrc

# Machines suivantes (snap et prepare-node-script déjà passés)
sunbeam cluster add noeud2.exemple.lan \
  --output noeud2.jeton      # sur la première
cat noeud2.jeton | sunbeam cluster join \
  --role control,compute,storage -   # sur la nouvelle
sunbeam cluster resize                  # passe en HA à 3

# Vie du cloud
sunbeam enable observability
sunbeam cluster refresh          # correctifs de la piste
```

Sujet	Ce que propose Canonical
Versions à long terme	2024.1 et 2026.1, alignées sur Ubuntu LTS
Durée de maintenance	5 ans en standard ; jusqu'à 12 avec Ubuntu Pro (+5) et Legacy Support (+2)
Support	abonnement Ubuntu Pro, par nœud
Offre gérée	exploitation par Canonical, sur votre matériel
Fonctions optionnelles	`sunbeam enable` : supervision, équilibrage de charge, orchestration, validation…

- **Même outil de un à plusieurs centaines de nœuds :** on commence sur une machine, on ajoute des nœuds sans réinstaller.
- **Mises à jour dans une piste :** `cluster refresh` applique les révisions du canal ; passer d'une piste à l'autre (2024.1 → 2026.1) n'est pas pris en charge par cette procédure.
- **Diagnostic :** `juju status` donne l'état de chaque charm, `kubectl` celui des pods, `snap logs` celui des services sur l'hôte.

> Les durées et le contenu des offres évoluent : à confirmer sur le site de l'éditeur au moment du choix.

 Approfondi au module 6

Sunbeam en production : l'infrastructure physique

Comment lire

Badge rouge
nombre de nœuds identiques

Point de couleur
interface du nœud sur ce réseau, un espace MAAS

VIP des API
plages internal-api et public-api, servies par MetalLB sur les nœuds control

Gouvernance
hors du cloud : MAAS, Juju et Sunbeam tournent en VM sur les Governor

Petit cloud
control, compute et storage sur 3 nœuds, fournisseur manuel, sans MAAS

Zone d'administration — 3 serveurs physiques « Governor » (Ubuntu 24.04) qui hébergent des machines virtuelles

governor-01 (hyperviseur KVM)	governor-02 (hyperviseur KVM)	governor-03 (hyperviseur KVM)
MAAS région + rack	MAAS région + rack	MAAS région + rack
Contrôleur Juju	Contrôleur Juju	Contrôleur Juju
Contrôleur Sunbeam	Contrôleur Sunbeam	Contrôleur Sunbeam
Client · snap openstack		

MAAS installe Ubuntu sur chaque nœud du cloud · le contrôleur Juju y pose un agent et déploie les charms

Nœud control ×3
- **snap k8s** Canonical Kubernetes
  - keystone, nova, neutron
  - glance, cinder, placement
  - horizon, traefik, mysql
  - rabbitmq, ovn-central, vault (opt.)
  - pods (rocks) pilotés par les charms
- agent Juju
- Ubuntu Server 24.04 LTS

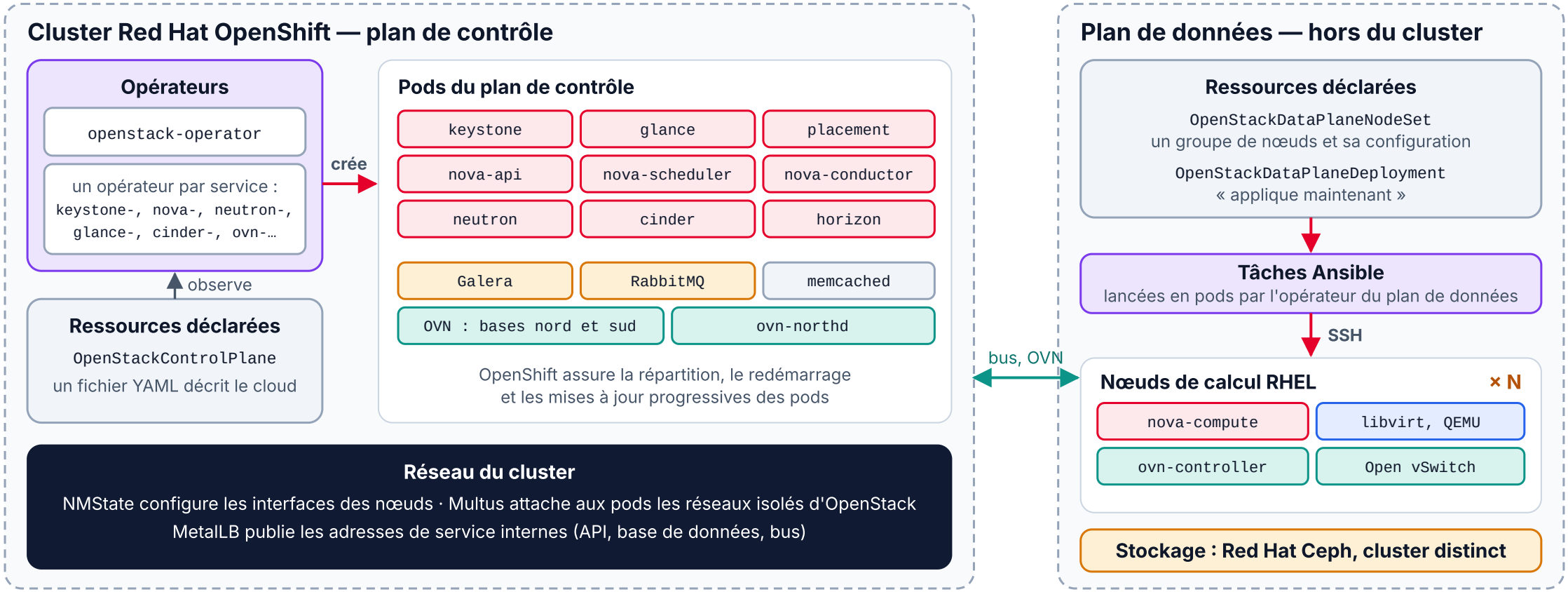
Nœud network ×2+
- **snap microovn**
  passerelle OVN : routeurs virtuels, SNAT, IP flottantes, sortie vers l'externe
- jamais avec le rôle compute
- agent Juju
- Ubuntu Server 24.04 LTS

Nœud compute ×N
- VM, VM, VM
- **snap openstack-hypervisor**
  nova-compute
  libvirt, QEMU/KVM
  ovn-controller, Open vSwitch
  agent de métadonnées
- agent Juju
- Ubuntu Server 24.04 LTS

Nœud storage ×3+
- **snap microceph**
  MON, MGR, un OSD par disque
  OSD, OSD, OSD
- **snap cinder-volume**
  volumes Cinder sur Ceph
- agent Juju
- Ubuntu Server 24.04 LTS

management
internal · API
public · API
data · Geneve
storage · Ceph
storage-cluster
external · physnet1

Red Hat OpenStack Services on OpenShift (RHOSO)



À retenir : le plan de contrôle est une application OpenShift décrite par des ressources Kubernetes ; les hyperviseurs restent des serveurs RHEL classiques, configurés par Ansible — mais Ansible est déclenché par un opérateur, plus par un administrateur.

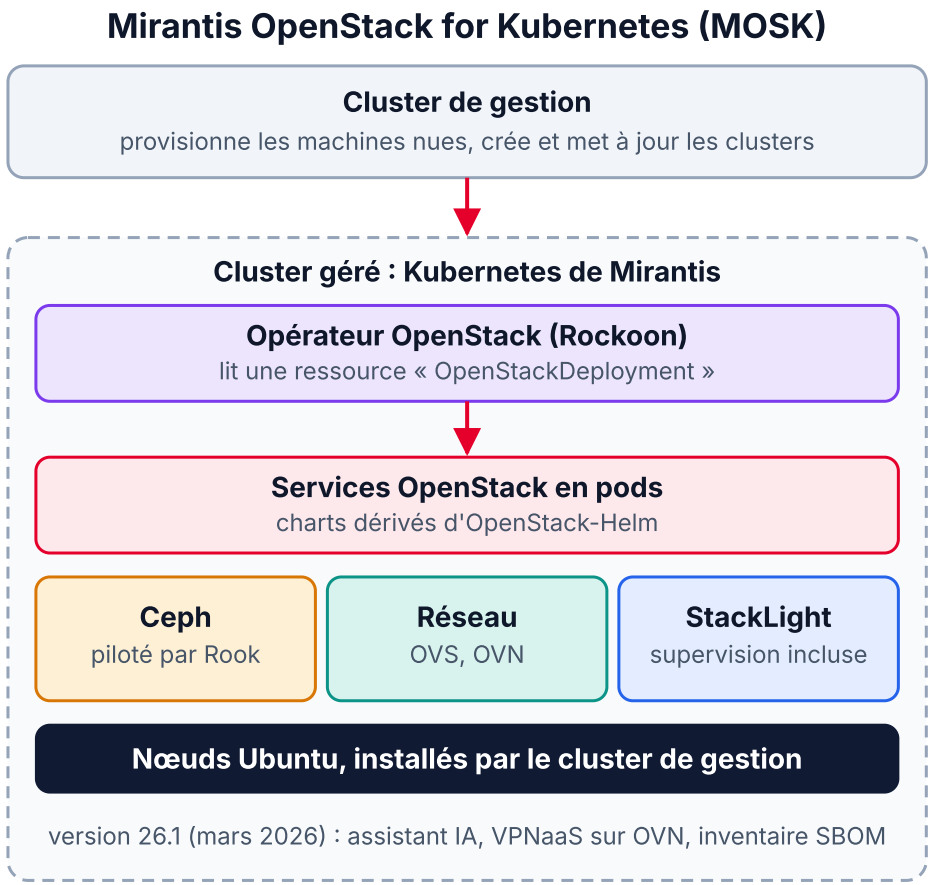
Red Hat : de director à OpenShift

Sujet	RHOSO 18 — génération actuelle	RHOSP 17.1 — génération précédente
Version d'OpenStack	**fondée sur 2023.1 « Antelope »**	fondée sur « Wallaby »
Outil de déploiement	**opérateurs sur OpenShift**	director, issu du projet TripleO (retiré en amont)
Prérequis	**un cluster OpenShift, trois nœuds au minimum**	une machine « undercloud » qui déploie l'« overcloud »
Plan de contrôle	**pods ; haute disponibilité assurée par OpenShift**	trois contrôleurs, conteneurs Podman, Pacemaker
Description du cloud	**ressources personnalisées Kubernetes**	gabarits Heat et fichiers d'environnement
Nœuds de calcul	**RHEL, configurés par Ansible à la demande d'un opérateur**	RHEL, configurés par Ansible depuis l'undercloud
Machines nues	**Metal3, intégré à OpenShift**	Ironic de l'undercloud
Mises à jour	**progressives, pilotées par les opérateurs**	procédure longue, en plusieurs phases
Passage de l'une à l'autre	« adoption » : le plan de contrôle est recréé sur OpenShift à partir des bases existantes ; les nœuds de calcul et les machines virtuelles sont conservés	

Ce que cela exige : des compétences OpenShift dans l'équipe d'exploitation, et un cluster OpenShift à maintenir en plus du cloud. La souscription couvre les deux.

Pourquoi ce changement : le projet amont TripleO a été retiré. Red Hat a remplacé un outil complexe et spécifique par les mécanismes standard de Kubernetes, déjà maîtrisés pour ses autres produits.

Mirantis et les autres éditeurs



Éditeur	Offre	Socle technique
Canonical	Canonical OpenStack, Charmed OpenStack	Juju, Kubernetes, snaps
Red Hat	RHOSO	opérateurs sur OpenShift
Mirantis	MOSK	Kubernetes, opérateur, charts Helm
VEXXHOST	Atmosphere, avec support ou hébergé	Kubernetes, charts Helm
Rackspace	OpenStack exploité par Rackspace	Genestack
Wind River	plateforme pour la périphérie	StarlingX
Platform9	plan de contrôle fourni en service	propre à l'éditeur

- **Tendance commune :** tous les éditeurs ont déplacé le plan de contrôle sur Kubernetes ces dernières années.
- **Intégrateurs :** beaucoup de clouds en production reposent sur Kolla-Ansible ou OpenStack-Ansible, avec le support d'une société de services spécialisée plutôt que d'un éditeur.
- **Éditeurs sortis du marché :** SUSE a arrêté sa distribution ; un éditeur peut se retirer, d'où l'intérêt d'un socle ouvert.

> Catalogue et noms d'offres changeants : à vérifier auprès de chaque éditeur.

Qui fait quoi : cinq modes de consommation

	Outil communautaire Kolla-Ansible, OSA, Helm	**Distribution supportée** Canonical, Red Hat, Mirantis	**Cloud privé géré** sur votre site, exploité par un tiers	**Cloud privé hébergé** matériel dédié chez un tiers	**Cloud public OpenStack** OVHcloud, Infomaniak, Cleura…
Matériel, centre de données	vous	vous	vous	fournisseur	fournisseur
Conception de l'architecture	vous	vous, guidé par l'éditeur	fournisseur	fournisseur	fournisseur
Déploiement	vous	vous, outil de l'éditeur	fournisseur	fournisseur	fournisseur
Correctifs du code	vous, depuis l'amont	éditeur	fournisseur	fournisseur	fournisseur
Montées de version	vous	vous, procédure de l'éditeur	fournisseur	fournisseur	fournisseur
Supervision, astreinte	vous	vous	fournisseur	fournisseur	fournisseur
Projets, quotas, images	vous	vous	vous	vous	fournisseur
Applications, données	vous	vous	vous	vous	vous

Lecture : de gauche à droite, on échange de la maîtrise et du temps d'ingénieur contre une facture et une dépendance. Le logiciel, lui, reste le même : les API sont identiques dans les cinq colonnes.

Avant de choisir un outil, choisir une colonne. Une équipe de trois personnes qui prend la première colonne consacrera l'essentiel de son temps à l'exploitation de la plateforme, pas aux utilisateurs.

Communautaire ou éditeur : ce que l'on achète vraiment

Une souscription n'achète pas le logiciel — il est libre. Elle achète quatre services, qu'il faut sinon produire en interne.

Correctifs rétroportés

Les failles et anomalies sont corrigées dans **votre** version, pendant des années, sans vous imposer de monter de version pour un correctif.

Combinaisons certifiées

Matériel, pilotes de stockage et de réseau, systèmes invités : une matrice testée, et un engagement de l'éditeur si elle ne fonctionne pas.

Durée de vie longue

Une version amont est maintenue environ dix-huit mois. Un éditeur la maintient plusieurs années : moins de montées de version, mieux préparées.

Un responsable à appeler

Délai d'intervention contractuel, escalade jusqu'aux développeurs, et un seul interlocuteur pour le système, Ceph et OpenStack.

Ce que l'on cède en échange

- **Le rythme :** les nouveautés arrivent quand l'éditeur les intègre, parfois plusieurs versions après l'amont.
- **Les choix techniques :** système hôte, réseau virtuel et stockage sont ceux que l'éditeur supporte.
- **La liberté de modifier :** un correctif maison ou un pilote non certifié peut faire sortir du support.
- **Le budget :** coût récurrent, généralement par nœud ou par processeur.

Trois questions pour trancher

- Avez-vous, en interne, quelqu'un capable de **lire le code** d'un service et de proposer un correctif en amont ?
- Pouvez-vous tenir un rythme d'**une montée de version par an** au minimum ?
- Une réglementation ou un client vous impose-t-il un **support contractuel** ?

> Deux « non » puis un « oui » : une distribution supportée ou une offre gérée s'impose. La voie intermédiaire existe : outil communautaire, plus un contrat avec un intégrateur.

05

Comparaison transversale

Les mêmes questions posées à toutes les méthodes : sur quoi reposent-elles, comment se comportent-elles en exploitation, que demandent-elles à l'équipe ?

- Fiche technique comparée
- Compétences requises
- Personnalisation
- Intégration du stockage
- Tailles et topologies
- Opérations courantes
- Montées de version
- Fonctionnement déconnecté
- Supervision et validation

5 · COMPARAISON TRANSVERSALE Sunbeam face aux autres : module 6

Fiche technique comparée

	Kolla-Ansible	OpenStack-Ansible	OpenStack-Helm	Sunbeam	RHOSO	MOSK
Statut	projet officiel, communautaire	projet officiel, communautaire	projet officiel, communautaire	projet officiel ; support Canonical	produit Red Hat, sous souscription	produit Mirantis, sous souscription
Système hôte	Ubuntu, Debian, Rocky ; CentOS Stream sans images	Ubuntu, Debian, CentOS Stream, Rocky	Ubuntu 24.04 testé	Ubuntu LTS	RHEL, RHCOS	Ubuntu
Réseau virtuel	OVN ou OVS, au choix	OVN par défaut	OVN ou OVS	OVN	OVN	OVN ou OVS
Ceph	externe, à déployer à part	externe, ou rôles intégrés	Rook, dans le cluster	MicroCeph, inclus ; quelques baies tierces	Red Hat Ceph, externe	Rook, inclus
Machines nues	avec Kayobe	non	non	avec MAAS	Metal3	inclus
Haute disponibilité	HAProxy, keepalived, Galera	HAProxy, keepalived, Galera	réplicas, services Kubernetes	réplicas, Traefik	réplicas, MetalLB, Galera	réplicas, services Kubernetes
Taille minimale	une machine (tout-en-un)	une machine (tout-en-un)	un cluster à un nœud	une machine	cluster OpenShift compact et un nœud de calcul	cluster de gestion et cluster géré

Lecture : les trois colonnes de gauche sont des outils que l'on assemble soi-même ; les trois de droite sont des ensembles intégrés où l'outil choisit pour vous le système, le réseau et le stockage. Orchestrateur, socle et forme de livraison : tableau des trois axes, partie 1. Valeurs relevées dans les documentations en octobre 2026.

5 · COMPARAISON TRANSVERSALE

Opérations courantes : la même tâche, cinq façons

Opération	Kolla-Ansible	OpenStack-Ansible	OpenStack-Helm	Sunbeam	RHOSO
Changer une option d'un service	fichier dans `/etc/kolla/config`, puis `reconfigure --tags`	variable de surcharge, puis rejouer le rôle du service	fichier de valeurs, puis `helm upgrade`	manifeste (`software.charms`), puis `cluster refresh --manifest`	modifier la ressource du plan de contrôle
Ajouter un hyperviseur	l'ajouter à l'inventaire, `deploy --limit`	l'ajouter au fichier de configuration, rejouer avec `--limit`	joindre le nœud au cluster, l'étiqueter	`cluster add`, puis `cluster join`	l'ajouter au groupe de nœuds, créer un déploiement
Redémarrer un service	`docker restart nova_api`	`systemctl restart` dans le conteneur LXC	`kubectl rollout restart`	`kubectl`, ou `snap restart` sur l'hôte	`oc rollout restart`
Lire les journaux	`/var/log/kolla`, `docker logs`	`journalctl` dans le conteneur	`kubectl logs`	`kubectl logs`, `snap logs`	`oc logs`
Sauvegarder la base	commande intégrée `mariadb-backup`	à organiser	tâche planifiée du chart MariaDB	action `create-backup` du charm MySQL, vers S3	procédure de l'éditeur
Remplacer un contrôleur	réinstaller l'hôte, `deploy --limit`, rejoindre Galera	recréer les conteneurs de l'hôte par les rôles	remplacer le nœud Kubernetes ; les pods sont replanifiés	retirer puis rejoindre le nœud	remplacer le nœud OpenShift

Outils Ansible : rien ne se passe tant que personne ne lance une commande. L'état réel peut dériver de l'état décrit sans que l'outil le sache.

Outils Kubernetes : un contrôleur compare en continu l'état réel à l'état décrit et corrige. Une modification manuelle est annulée en quelques secondes.

5 · COMPARAISON TRANSVERSALE

Compétences à réunir dans l'équipe

	Linux, réseau	OpenStack	Ansible	Conteneurs	Kubernetes	Outil propre à la méthode	Construction d'images
Kolla-Ansible	●	●	●	● Docker, Podman	—	○ Kolla	●
OpenStack-Ansible	●	●	●	◐ LXC	—	○ rôles OSA	●
OpenStack-Helm	●	●	○	◐ pods	●	◐ Helm, charts	●
Sunbeam	◐	●	—	○ pods, snaps	◐	● Juju, snaps	—
RHOSO	◐	●	◐	○ pods, Podman	●	● OpenShift, opérateurs	—
MOSK	◐	●	—	○ pods	●	◐ opérateur Mirantis	—

● indispensable au quotidien · ◐ nécessaire pour dépanner · ○ utile, sollicité à l'occasion · — non requis

Constante : aucune méthode ne dispense de comprendre Linux, le réseau et OpenStack lui-même. L'outil automatise l'installation, pas le diagnostic d'une instance qui ne démarre pas.

Variable : la technologie d'orchestration. Appréciation qualitative, à adapter : une équipe qui exploite déjà Kubernetes n'a pas à l'apprendre pour OpenStack-Helm.

Montées de version : le vrai test d'une méthode



Une version sort tous les six mois. Depuis 2023.1, les versions « .1 » sont dites SLURP : on peut passer de l'une à la suivante sans s'arrêter sur la version intermédiaire.

Ce que fait toute montée de version

- **Plan de contrôle d'abord :** nouveau code, migration du schéma des bases, redémarrage des API une à une.
- **Nœuds de calcul ensuite**, par lots : les agents acceptent de dialoguer avec un contrôleur plus récent qu'eux.
- **Les machines virtuelles continuent de tourner :** seule l'API est brièvement perturbée, pas le trafic des instances.
- **Pas de retour en arrière simple :** une fois le schéma migré, revenir exige de restaurer la sauvegarde.

Méthode	Ce qui change concrètement
Kolla-Ansible	nouvelle version de l'outil, nouvelles images, `upgrade`
OpenStack-Ansible	nouvelle étiquette git, environnements reconstruits
OpenStack-Helm	nouveaux charts et images, `helm upgrade` ordonné
Sunbeam	`cluster refresh` dans une piste ; pas de passage d'une piste à l'autre à ce jour
RHOSO, MOSK	l'opérateur enchaîne les étapes après validation

À vérifier pour chaque outil : le saut SLURP est-il supporté par **l'outil**, et la version du système hôte doit-elle changer en même temps ?

5 · COMPARAISON TRANSVERSALE

Personnalisation : jusqu'où peut-on s'écarter du standard ?

Niveau 1 — Une option
Changer une valeur dans un fichier de configuration d'un service. **Toutes** les méthodes le permettent, avec plus ou moins d'options exposées.

Niveau 2 — Un pilote tiers
Baie de stockage, équipement réseau : le pilote doit être **présent dans l'image** ou le paquet. Souvent une image à reconstruire.

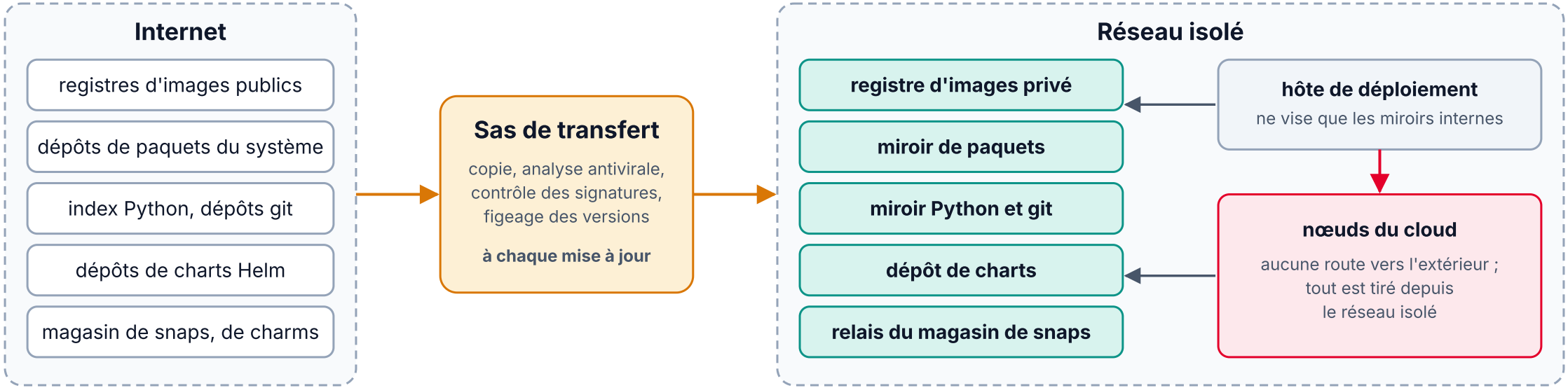
Niveau 3 — Un correctif de code
Embarquer un correctif pas encore publié. Facile quand l'outil installe depuis les sources, plus lourd avec des images figées.

Niveau 4 — L'architecture
Changer de répartiteur, de base, de topologie. Revient à **modifier l'outil** : à éviter, ou à contribuer en amont.

Méthode	Régler une option de Nova	Embarquer un correctif ou un pilote	Liberté
Kolla-Ansible	fichier `/etc/kolla/config/nova.conf`	reconstruire l'image avec `kolla-build`, la publier dans son registre	élevée
OpenStack-Ansible	variable `nova_nova_conf_overrides`	pointer le dépôt et la branche du service vers les siens	très élevée
OpenStack-Helm	clé `conf.nova` du fichier de valeurs	reconstruire l'image LOCI, changer l'étiquette dans les valeurs	élevée
Sunbeam	manifeste : `software.charms.nova-k8s.config`, puis `cluster refresh --manifest`	attendre la publication dans le canal ; pilotes limités à ceux que prévoit l'éditeur	limitée
RHOSO	champ `customServiceConfig` de la ressource	images certifiées ; correctif par le support de l'éditeur	encadrée

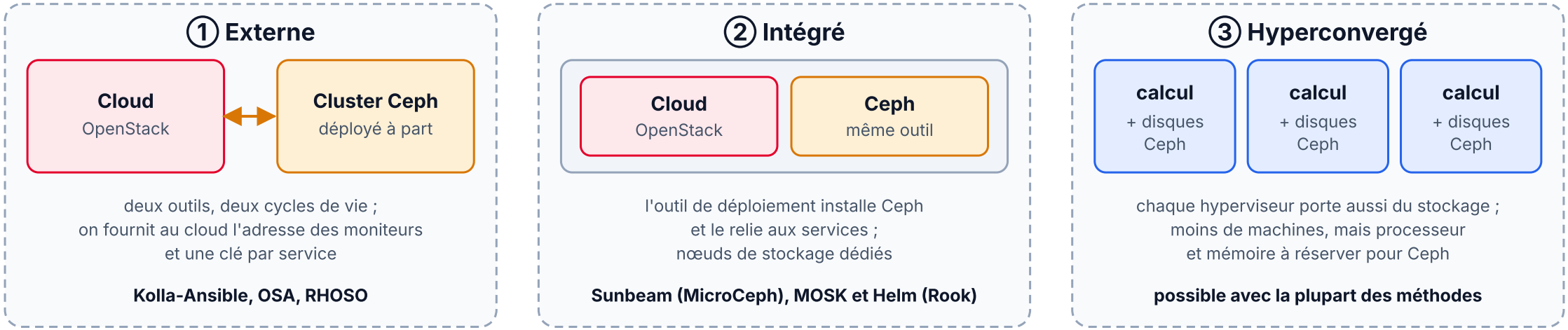
Règle de prudence : chaque écart au standard est une ligne de plus à vérifier à chaque montée de version. Tenir un registre des écarts, dans le même dépôt que la configuration.

Déployer sans accès à Internet



Méthode	Ce qu'il faut rendre disponible en interne	Difficulté
Kolla-Ansible	les images Kolla, les paquets du système, les dépendances Python de l'outil	modérée : une seule variable désigne le registre
OpenStack-Ansible	miroirs de paquets, de l'index Python et des dépôts git de chaque service	élevée : de nombreuses sources à refléter
OpenStack-Helm	images, charts, et tout ce qu'exige déjà le cluster Kubernetes	modérée si le cluster est déjà déconnecté
Sunbeam	relais du magasin de snaps, registre pour les rocks, charms	mandataire pris en charge ; hors ligne complet pas encore disponible
RHOSO, MOSK	registre miroir alimenté par les outils de l'éditeur	procédure documentée et supportée

Stockage : trois façons d'intégrer Ceph



Ce que l'outil doit toujours faire, quel que soit le mode

- Distribuer `ceph.conf` et les **clés** à Glance, Cinder et Nova.
- Déclarer le **secret libvirt** sur chaque hyperviseur, pour que QEMU ouvre les volumes.
- Configurer les **pools** attendus : images, volumes, sauvegardes, disques éphémères.

Et quand le stockage n'est pas Ceph

- **Baie du commerce :** vérifier que le pilote Cinder est dans l'image ou le paquet livré, et que l'éditeur le supporte.
- **LVM local :** suffisant pour une maquette ; un seul nœud porte les volumes, sans redondance.
- **Réseau de stockage :** à dimensionner et à isoler quelle que soit la méthode.

5 · COMPARAISON TRANSVERSALE

Supervision et validation : ce qui est fourni

Méthode	Journaux centralisés	Métriques et tableaux de bord	Validation du cloud
Kolla-Ansible	**Fluentd vers OpenSearch, à activer**	Prometheus et Grafana, à activer	à lancer soi-même (Tempest, Rally)
OpenStack-Ansible	**journald sur chaque hôte ; centralisation à votre charge**	à votre charge	rôle Tempest fourni
OpenStack-Helm	**charts de collecte fournis, à assembler**	charts Prometheus et Grafana fournis	tests Helm par chart, Tempest en pod
Sunbeam	**Loki, par `sunbeam enable observability`**	Prometheus et Grafana, même commande	`sunbeam enable validation`
RHOSO	**journalisation d'OpenShift**	opérateur de télémétrie, supervision d'OpenShift	opérateur de tests
MOSK	**StackLight, inclus**	StackLight, inclus	tests intégrés au produit

Tempest

La suite de tests officielle : elle appelle les API comme un utilisateur et vérifie les réponses. Sert à prouver qu'un cloud tout juste déployé, ou tout juste mis à jour, fonctionne.

Rally

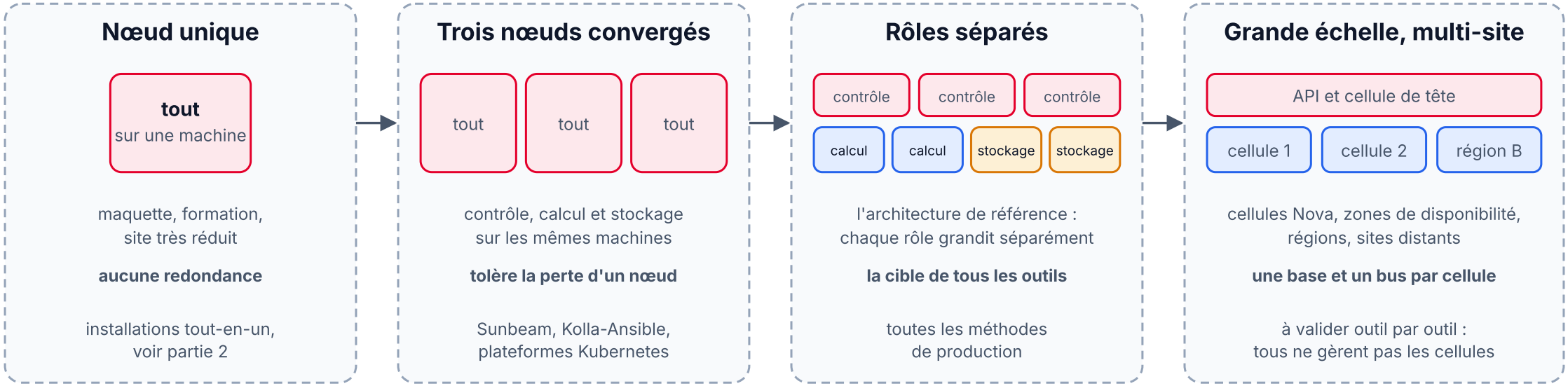
Mesure le comportement sous charge : combien d'instances par minute, en combien de temps, avec quel taux d'échec. Utile pour comparer avant et après un changement.

À prévoir dans tous les cas

Alertes sur la base de données, le bus de messages et les certificats ; sonde externe qui crée réellement une instance à intervalle régulier.

Critère souvent oublié : une méthode qui livre la supervision déjà branchée fait gagner des semaines. Sinon, compter ce chantier dans le projet.

Tailles et topologies : du nœud unique au multi-site



- **La méthode doit couvrir la taille visée, pas seulement la taille de départ.** Passer de trois à trente nœuds ne doit pas imposer de changer d'outil.
- **Ce qui limite en premier :** la durée d'une exécution complète de l'outil, qui croît avec le nombre d'hôtes ; puis la base de données et le bus de messages.
- **Sites distants :** vérifier si l'outil sait déployer des nœuds de calcul éloignés du plan de contrôle, et ce qui se passe quand le lien tombe.
- **Plusieurs clouds :** un dépôt de configuration par cloud, un tronc commun partagé — possible avec tous les outils pilotés par fichiers.

06

Critères de choix

Il n'existe pas de meilleure méthode, seulement une méthode adaptée à une équipe, à un besoin et à un horizon de temps. Voici comment la trouver.

- Dix critères à instruire
- Durée de vie et support
- Arbre de décision
- Grille de notation et maquette
- Le critère dominant : les compétences
- Coût complet
- Profils types
- Erreurs fréquentes

6 · CRITÈRES DE CHOIX

Dix critères à instruire avant de choisir

1 · Compétences
Ce que l'équipe sait déjà dépanner : Ansible, Kubernetes, Juju, OpenShift.

2 · Taille de l'équipe
Combien de personnes, et quelle part de leur temps pour la plateforme.

3 · Support
Contrat exigé, ou entraide communautaire acceptable.

4 · Durée de vie
Fréquence des montées de version que l'on peut réellement tenir.

5 · Budget
Souscriptions d'un côté, salaires et formation de l'autre.

6 · Échelle
Nombre de nœuds aujourd'hui et dans trois ans ; un site ou plusieurs.

7 · Existant
Système d'exploitation imposé, Kubernetes déjà en place, outillage maison.

8 · Personnalisation
Pilotes tiers, correctifs maison, architecture atypique.

9 · Contraintes
Réseau isolé, certification, souveraineté, matériel imposé.

10 · Réversibilité
Peut-on changer d'éditeur ou d'outil sans tout reconstruire ?

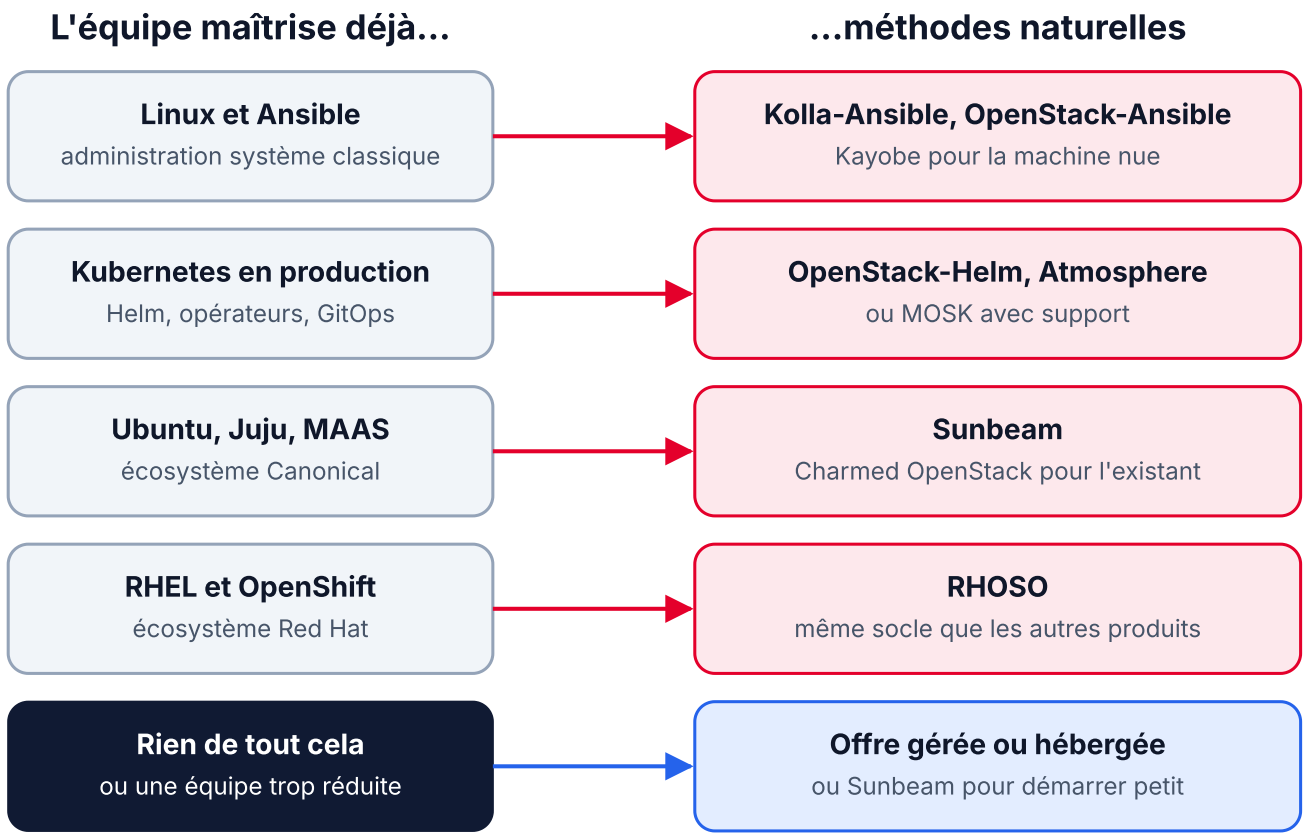
Critères liés à l'organisation (1 à 5)

- Ils pèsent **plus lourd** que les critères techniques : toutes les méthodes de production savent déployer un cloud correct.
- Ils sont **stables** : une équipe ne change pas de culture technique en six mois.
- Ils se vérifient par des questions simples, pas par une maquette.

Critères liés au projet (6 à 10)

- Ils servent surtout à **éliminer** : une contrainte non satisfaite écarte une méthode, quelle que soit sa note par ailleurs.
- Ils se vérifient par la **documentation**, puis par une maquette.
- Ils évoluent : les réévaluer à chaque étape importante du projet.

Le critère dominant : ce que l'équipe sait dépanner



Point de départ, pas verdict : les autres critères peuvent le remettre en cause.

Le test décisif : un incident survient en pleine nuit. La personne d'astreinte sait-elle où regarder **sans ouvrir la documentation de l'outil** ?

- **L'outil devient une compétence à part entière.** On ne dépanne pas Nova dans Kubernetes comme Nova sur un serveur : il faut d'abord trouver le pod, son journal, sa configuration.
- **Deux technologies à apprendre en même temps** — OpenStack et son orchestrateur — est le premier facteur d'échec des projets.
- **Compter les personnes, pas l'équipe :** si une seule personne connaît l'outil, le cloud dépend de ses congés.
- **Recruter :** les profils Ansible et Kubernetes sont nombreux ; les profils Juju ou spécialistes d'un outil précis sont plus rares.

Former l'équipe à l'orchestrateur **avant** le projet, pas pendant la mise en production.

Durée de vie et support : combien de temps une version tient-elle ?

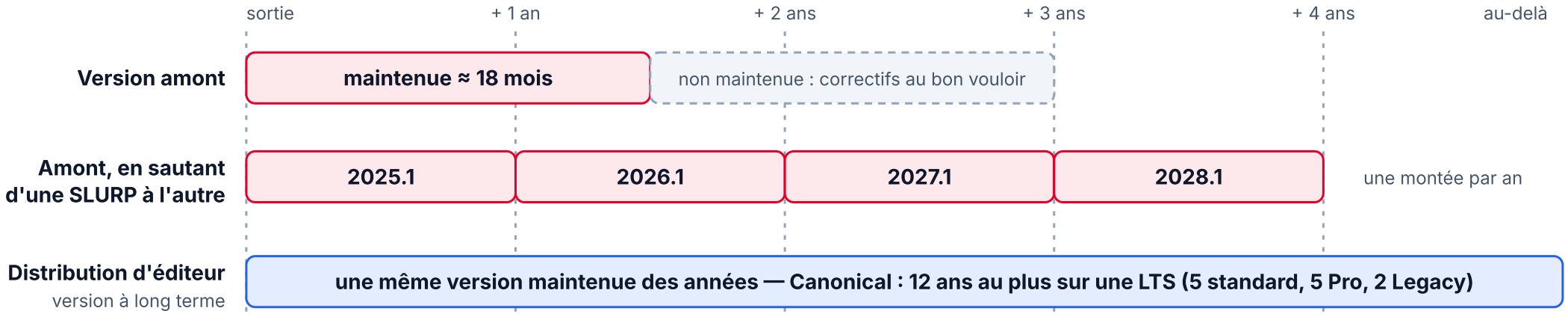


Schéma de principe : les durées exactes dépendent de chaque version et de chaque contrat.

Suivre l'amont

Une montée de version tous les six mois, ou une par an en ne prenant que les versions SLURP. Exige une chaîne de test et une équipe rodée. En retour : les nouveautés tout de suite, aucun coût de licence.

S'appuyer sur un éditeur

Une version garde ses correctifs de sécurité pendant des années. Les montées de version deviennent rares, mais chacune franchit plusieurs versions amont : à préparer comme un projet.

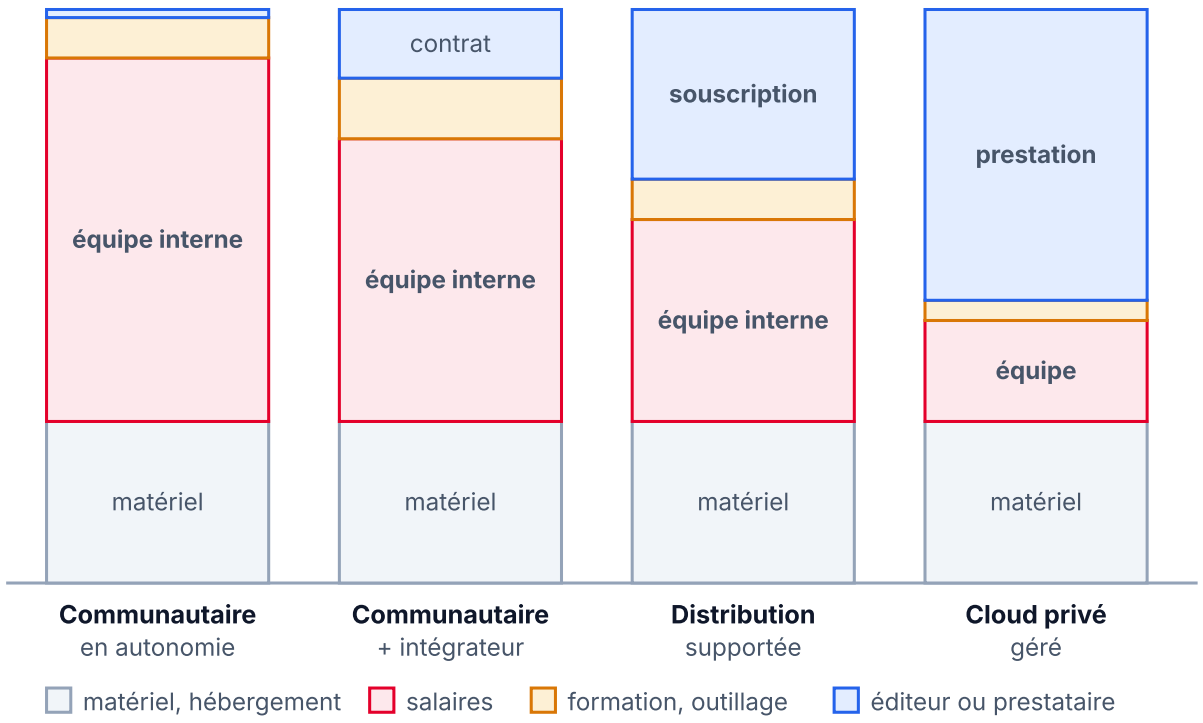
Le piège : ne rien faire

Un cloud communautaire laissé trois ans sans mise à jour n'a plus de correctifs et ne peut plus monter que version par version. Rattraper le retard coûte plus cher que de le prévenir.

Question à poser en comité : « Qui, nommément, fera la prochaine montée de version, et quand ? » Sans réponse, privilégier une version à long terme sous contrat.

Coût complet : où va l'argent selon le mode choisi

Répartition illustrative des postes de coût sur cinq ans — proportions de principe, pas des chiffres mesurés



Ce que le graphique veut montrer

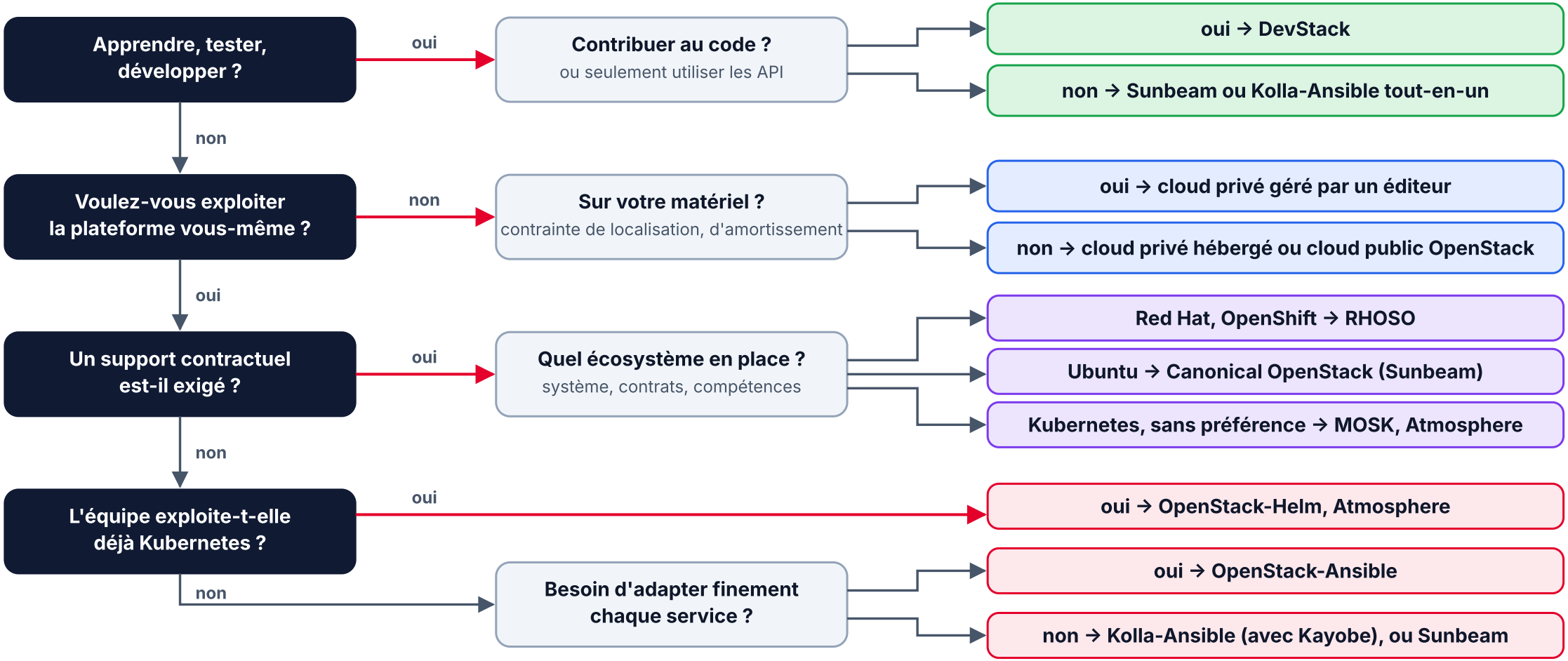
- **Le matériel ne dépend pas de la méthode.** Il représente pourtant souvent le premier poste : la méthode se joue sur le reste.
- **« Gratuit » déplace le coût vers les salaires.** Un cloud communautaire en autonomie exige une équipe plus nombreuse et plus expérimentée.
- **Une souscription ne supprime pas l'équipe :** il faut toujours des personnes pour exploiter, même avec un éditeur.
- **Le total n'est pas forcément différent.** Ce qui change, c'est la part fixe, la part variable et qui porte le risque.

Coûts que l'on oublie

- L'environnement de **préproduction**, indispensable pour répéter les montées de version.
- La **rotation du personnel** : former un remplaçant sur l'outil prend des mois.
- Le coût d'une **sortie** : migrer les charges si l'éditeur ou l'outil disparaît.

> Demander des devis réels et chiffrer ses propres salaires : aucun ordre de grandeur générique ne vaut pour votre contexte.

Arbre de décision



Mode d'emploi : l'arbre donne une **première** piste en moins d'une minute. Il ne remplace ni la grille de notation ni la maquette : une contrainte forte (réseau isolé, matériel imposé) peut invalider la branche obtenue.

6 · CRITÈRES DE CHOIX

Profils types : quelle méthode pour quel contexte ?

Contexte	Piste principale	Alternative	Pourquoi
Formation, découverte des API	**Sunbeam sur une machine**	Kolla-Ansible tout-en-un	installation courte, proche d'un vrai cloud
Développement d'OpenStack	**DevStack**	—	seul outil pensé pour modifier le code
Laboratoire, université	**Kolla-Ansible**	Sunbeam	gratuit, bien documenté, large communauté
PME, équipe de 2 à 4 personnes	**Sunbeam avec support**	offre gérée	peu de décisions à prendre, version à long terme
Hébergeur, cloud public régional	**Kolla-Ansible avec Kayobe**	OpenStack-Ansible, Atmosphere	maîtrise complète, pas de coût par nœud
Grande entreprise déjà sur OpenShift	**RHOSO**	MOSK	même socle, mêmes équipes, même contrat
Opérateur télécom, sites en périphérie	**distribution d'éditeur certifiée**	StarlingX	certification des fonctions réseau, sites nombreux
Secteur réglementé, réseau isolé	**distribution supportée**	Kolla-Ansible et registre local	procédure déconnectée documentée, engagement écrit
Équipe plateforme centrée Kubernetes	**OpenStack-Helm, Atmosphere**	MOSK	réutilise GitOps, supervision et savoir-faire

À lire comme des tendances, pas comme des règles. Chaque ligne suppose le contexte « typique » ; votre équipe, vos contrats en cours et vos contraintes peuvent inverser la recommandation.

6 · CRITÈRES DE CHOIX

Grille de notation pondérée et maquette

Critère	Poids	Méthode A	Méthode B	Méthode C
Compétences déjà présentes	5	4	2	3
Montées de version	4	4	4	5
Support disponible	3	2	2	5
Personnalisation	3	4	4	2
Fonctionnement déconnecté	2	4	3	4
Coût sur cinq ans	3	4	3	2
Total pondéré	**sur 100**	**74**	**59**	**70**

Exemple entièrement fictif, pour montrer la mécanique : notes de 1 à 5, multipliées par le poids, total ramené à 100. Les poids se fixent **avant** de noter, et par le commanditaire, pas par l'équipe technique seule.

- **Deux totaux proches** (ici A et C) : la grille ne tranche pas, la maquette le fera.
- **Un critère éliminatoire** ne se note pas : il écarte la méthode avant la grille.

La maquette : sept épreuves à faire passer

- **1** · Déployer trois contrôleurs et deux nœuds de calcul **depuis zéro**, en suivant uniquement la documentation.
- **2** · Changer une option de Nova et vérifier qu'elle est appliquée.
- **3** · Ajouter un nœud de calcul, puis en retirer un.
- **4** · Éteindre brutalement un contrôleur, constater, le réintégrer.
- **5** · **Monter de version**, avec des instances en fonctionnement.
- **6** · Restaurer la base de données depuis une sauvegarde.
- **7** · Tout détruire et redéployer : le résultat est-il identique ?

Ce que l'on mesure : le temps passé, le nombre de recours à l'aide extérieure, et surtout ce que l'équipe a **compris** de ce que faisait l'outil.

Huit erreurs fréquentes

Choisir sur la démonstration
Juger l'outil sur l'installation du premier jour, alors que le coût se joue sur les années d'exploitation.

Mettre DevStack en production
Ou tout autre outil de maquette « parce qu'il fonctionne déjà ». Aucune montée de version n'est prévue.

Suivre la mode
Adopter Kubernetes pour OpenStack sans équipe Kubernetes : deux plateformes à apprendre en même temps.

Modifier hors de l'outil
Corriger un fichier à la main sur un nœud. Le changement disparaît à la prochaine exécution, souvent au pire moment.

Oublier la machine nue
Choisir un outil qui commence « au système installé » sans avoir de moyen de réinstaller un serveur à l'identique.

Reporter les montées de version
Chaque version sautée rend la suivante plus risquée, jusqu'à imposer une reconstruction complète.

Ne pas avoir de préproduction
Répéter la montée de version directement en production. La maquette du projet doit devenir un environnement permanent.

Dépendre d'une seule personne
Un outil connu d'un seul ingénieur est un risque d'exploitation, quel que soit l'outil.

Les signes d'un bon choix

- Toute la configuration tient dans **un dépôt git**, relu et versionné.
- Un nouveau venu redéploie la préproduction **en suivant le dépôt**, sans aide.
- La dernière montée de version date de **moins d'un an**.

Changer d'avis coûte cher. Passer d'une méthode à une autre revient à construire un second cloud et à migrer les charges. Mieux vaut trois semaines de maquette que trois ans avec le mauvais outil.

SYNTHÈSE

Ce qu'il faut retenir

Le paysage

- **Apprendre :** DevStack, installations tout-en-un.
- **Communautaire :** Kolla-Ansible, OpenStack-Ansible, OpenStack-Helm.
- **Éditeurs :** Canonical (Sunbeam), Red Hat (RHOSO), Mirantis (MOSK).
- **Retirés :** TripleO, les recettes Chef — ne plus démarrer avec.

La grille de lecture

- **Six couches :** quelle part l'outil couvre-t-il ?
- **Trois axes :** forme de livraison, orchestrateur, socle d'exécution.
- **Cinq modes :** de l'autonomie complète au cloud public.
- **Tendance :** le plan de contrôle migre vers Kubernetes.

La démarche

- Choisir d'abord **qui exploite**, ensuite l'outil.
- Partir des **compétences** de l'équipe.
- Éliminer par les **contraintes**, classer par la grille.
- Trancher par une **maquette** incluant une montée de version.

Si vous ne retenez qu'une ligne	Méthode	En une phrase
Le plus direct	**Kolla-Ansible**	des conteneurs prêts à l'emploi, pilotés par Ansible ; très présent en production
Le plus ajustable	**OpenStack-Ansible**	tout depuis les sources, chaque service réglable et remplaçable
Le plus « natif Kubernetes »	**OpenStack-Helm et dérivés**	OpenStack comme une application du cluster, pour les équipes déjà outillées
Le plus simple à démarrer	**Sunbeam**	d'une machine à un cloud complet avec le même outil, support optionnel
Le plus encadré	**RHOSO, MOSK**	un produit, un contrat, un socle Kubernetes imposé

Sources : docs.openstack.org (guides Kolla-Ansible, OpenStack-Ansible, OpenStack-Helm, Sunbeam), governance.openstack.org, documentations Red Hat, Canonical, Mirantis et VEXXHOST — consultées en octobre 2026.